

MgcCT: Motif-Preserving Graph Clustering on Continuous-Time Temporal Graphs

Longlong Lin¹, Zeli Wang², Qiangqiang Dai³, Jin Zhao⁴, Rong-Hua Li³

¹Southwest University, Chongqing, China; ²Chongqing University of Posts and Telecommunications, Chongqing, China;

³Beijing Institute of Technology, Beijing, China; ⁴Huazhong University of Science and Technology, Wuhan, China;

longlonglin@swu.edu.cn; zlwang@cqupt.edu.cn; qiangd66@gmail.com; zjin@hust.edu.cn; lironghuabit@126.com

Abstract—Temporal graphs naturally model numerous complex real-life systems, including financial, social, and transportation networks. While temporal graph clustering has achieved significant success, neither continuous topological-temporal dependencies nor higher-order connectivity patterns have been well-explored, leading to sub-optimal performances. To alleviate these dilemmas, we introduce a novel problem called *Motif-preserving graph clustering on Continuous-time Temporal graphs (MgcCT)*. Namely, given a continuous-time temporal graph and a temporal motif, *MgcCT* aims to detect a high-quality cluster by preserving the connectivity of higher-order structures built up from the temporal motif. To this end, we first propose a new concept of temporal motif conductance, which can be interpreted as the probability of escape from a random walk in the cluster. Then, we formulate *MgcCT* as an optimization task that aims at finding a cluster with minimum temporal motif conductance. Subsequently, we prove that the *MgcCT* problem is NP-hard and propose two efficient algorithms. Specifically, we first develop an approximate global algorithm *AGA* and theoretically prove *AGA* can achieve the near-optimal quadratic Cheeger bound for an arbitrary temporal motif. To further boost efficiency, we then propose a heuristic local algorithm *HLA* without having to access the entire graph. Finally, extensive empirical results on eight real-world datasets showcase the effectiveness, efficiency, and scalability of our solutions compared to ten competitors.

I. INTRODUCTION

Graph clustering is a fundamental graph data mining task, which has been widely studied in image segmentation [1], [2], load balancing [3], and machine learning [4], [5]. Informally, graph clustering aims to partition the vertices of a graph into several non-overlapping clusters, satisfying the condition that the vertices within the same cluster have more edges than those in different clusters. However, real-life graphs often contain complex but useful time information (i.e., the edges are associated with rich timestamps), which enables providing profound insights into reasons and predicting their temporal behaviors [6], [7]. For example, in collaboration networks, the timestamps of the edges indicate when the corresponding authors co-authored papers. Therefore, many researchers are working on designing various temporal graph clustering algorithms to simultaneously consider the entanglement of structure and temporal dynamic [8]–[11].

Despite the significant success of temporal graph clustering, existing methods suffer from two-fold fatal defects. Firstly, they approximately compress the raw continuous-time temporal networks to the discrete time-aggregated snapshots, in which each snapshot consists of edges that occur in the user-

specified temporal resolution (e.g., one hour or day). Unfortunately, the discrete and non-continuous temporal graphs seriously affect the effectiveness of approaches [11]–[14]. Imagine that when the resolution is too coarse, it may not distinguish paths correctly, which is an important indicator for clustering [15]–[17]. Taking Fig. 1 as an example, (v_1, v_2, v_4) of (b) is an invalid path with respect to (w.r.t.) time-order in (a), and the valid path (v_1, v_2, v_5) of (a) cannot be identified by (b) and (c). Besides, it may also break some important temporal structures, such as burst events that occur over a short period of time. Therefore, these existing methods overlook important continuous topological-temporal dependencies and may identify sub-optimal results, as stated in our empirical results (Section V). Secondly, they adopt the dyadic temporal edge as the atomic clustering unit to partition the graph, while significantly ignoring the frequently occurring higher-order temporal connectivity patterns (a.k.a., temporal motifs [18], [19]). Actually, unlike temporal edges, each temporal motif is a small subgraph with temporal constraints (e.g., temporal ordering and duration) that indicates some unique functions [20]–[22]. Therefore, using such mesoscopic-level temporal motifs as the new clustering unit (i.e., each temporal motif should be regarded as a whole that cannot be split [18]) for temporal graph clustering has significant theoretical value and application scenarios but remains nascent. To name but a few, in temporal money transfer networks, the cycle structure that follows a chronological order with short duration (named temporal cycle) is an important factor in judging money laundering [23]–[25]. Thus, the clusters with dense temporal cycles can be used to identify financial criminal gangs. In temporal customer-product networks, the temporal butterfly (i.e., the (2,2)-biclique with temporal constraints) is a significant pattern of buying behavior [26]. Therefore, the clusters with dense temporal butterflies are suitable to enhance the accuracy and effectiveness of recommendations. Summing up, detecting motif-preserving clusters on continuous-time temporal networks can reveal some interesting phenomena, which has also been verified in our experiments (Section V).

Motif-preserving graph clustering (a.k.a., higher-order graph clustering) can typically be addressed by the following three potential models [27], [28]: motif modularity, motif density, and motif conductance. The first two models only consider the internal higher-order cohesiveness and overlook the important external sparsity, which is an important factor in measuring the

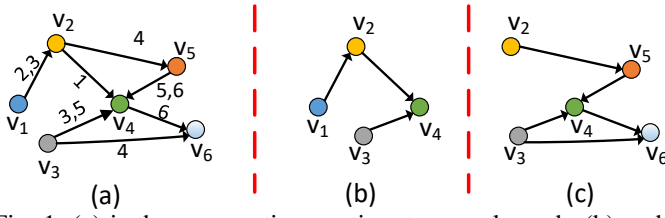


Fig. 1: (a) is the raw continuous-time temporal graph. (b) and (c) are time-aggregated snapshots by aggregating timestamps 1-3 and 4-6, respectively.

clustering quality [16], [29]–[32], resulting in unsatisfactory performance (Section V). This aspect, however, is addressed by the third model [33]–[39]. Inspired by this, we move beyond traditional motif conductance by integrating temporal information and formulating a novel concept of temporal motif conductance. Specifically, temporal motif conductance is the ratio of the total weight of temporal motif instances going out of the cluster to the total weight of temporal motif instances within the cluster (Definition 7). Smaller temporal motif conductance implies the partition preserves rich temporal motif instances inside the cluster and cuts temporal motif instances as little as possible. Besides, temporal motif conductance can be viewed as the escape probability of the cluster (Theorem 3.1), which generalizes the random walk interpretation of conventional conductance [34], [40]. Based on this, we formulate our problem of Motif-preserving graph clustering on Continuous-time Temporal graphs ($MgcCT$) as an optimization task that aims at finding a cluster with the minimum temporal motif conductance. Consequently, from the perspectives of both cut and escape probability, $MgcCT$ has strong interpretability (more advantages in Section II).

$MgcCT$ is practically important but technically challenging. For example, one simple treatment for solving $MgcCT$ is to utilize static higher-order graph clustering methods [37]–[39] to temporal networks by ignoring the time information of edges, but such solutions greatly compress the informative time semantics behind temporal graphs and thus obtain sub-optimal clustering qualities (Section V). Another possible method is to project the original temporal graph into a static weighted graph by enumerating the temporal motif and then adopting the classic graph clustering algorithm to partition the weighted graph. However, such a naive solution has only provable clustering qualities for the temporal motif consisting of three vertices [38], which limits real-world applications [19], [41]–[43]. Besides, it requires an exact enumeration of all temporal motif instances, which incurs prohibitively high time&space overheads (even is NP-hard for the simple temporal star [20]) due to the topological-temporal combinatorial explosions [18], [41]–[43]. On the other hand, as delineated in Section VI, many temporal lower-order graph clustering algorithms have been proposed in the literature, encompassing modularity-based [9], conductance-based [8], and density-based [11] methods. However, these approaches (as shown in Table I) overlook the more insightful higher-order substructures, ultimately leading to subpar empirical clustering

performance (Section V). In summary, existing methods suffer from sub-optimal clustering qualities, limited applications, and entail prohibitive computational costs.

To overcome these challenges, we first propose an approximate global algorithm AGA based on our proposed *edge-focused* weighted graph projection method. Specifically, we observe that the main reason why existing methods cannot obtain an accuracy guarantee is that the temporal motif cut (Definition 4) does not have a theoretical guarantee in the corresponding weighted graph (Section III-B). The explanation behind this phenomenon is that during the construction of the weighted graph, the weights of edges are obtained by enumerating hyperedges (i.e., temporal motifs), resulting in the loss or magnification of the original edge information. To recover this information, AGA specifies that an edge’s weight is non-zero iff it also exists as an edge in the original temporal graph (Definition 10). Surprisingly, we theoretically prove that this simple yet important adjustment achieves the near-optimal quadratic Cheeger bound for an arbitrary temporal motif, which is our main theoretical contribution. Additionally, unlike existing methods that exhaustively enumerate all temporal motif instances, AGA leverages highly efficient combinatorial counting techniques to directly compute the edge weights, significantly reducing computational costs. To further boost the efficiency, we devise a heuristic local algorithm, HLA , which does not require accessing the entire graph. Specifically, HLA starts a random walk from a query node to obtain a *local* probability distribution π (i.e., π has very few non-zero elements) by dynamically computing the edge weights on the fly. This approach greatly reduces the time and space overheads by avoiding the need to enumerate unnecessary temporal instances in advance. Then, HLA executes a *sweep-cut* procedure over π to obtain a local optimal heuristic result. In short, our main contributions are summarized as follows.

- **A Novel Model.** We introduce a novel problem of Motif-preserving graph clustering on Continuous-time Temporal graphs ($MgcCT$) and prove $MgcCT$ is the minimum escape probability of a random walk from the cluster, enhancing the interpretability of our model.
- **Two Efficient Algorithms.** We theoretically prove the NP-hardness of our problem and propose two efficient algorithms. Specifically, we first devise an approximation global algorithm AGA , which can achieve the near-optimal quadratic Cheeger bound for any temporal motif. Then, a heuristic local algorithm HLA is proposed to balance efficiency and quality.
- **Extensive Experiments.** Comprehensive empirical results on eight real-world datasets demonstrate our proposed solutions indeed are efficient and effective when contrasted with ten competitors. Notably, our algorithms outperform the baselines by 1~4 orders of magnitude in speed with higher clustering quality, alongside a notable reduction in memory usage of up to an order of magnitude.
- **Reproduction.** The source code of this paper can be found at <https://anonymous.open.science/r/MgcCT-B04F>.

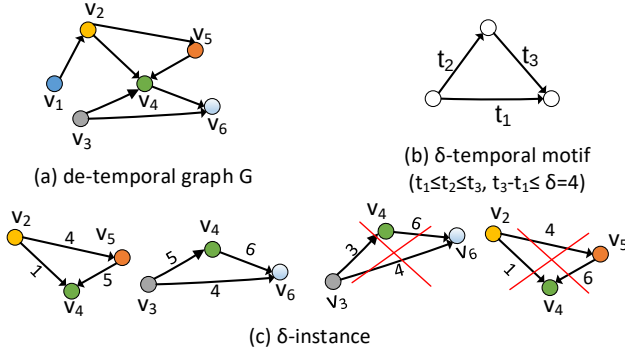


Fig. 2: The latter two of (c) are not δ -instances because they respectively violate conditions (i) and (ii) in Definition 2.

II. PRELIMINARIES

A. Basic Notations

Let $\mathcal{G}(V, \mathcal{E})$ be a directed continuous-time temporal graph, where V denotes the node set and $\mathcal{E}$ represents the directed temporal edge set. Each temporal edge $(u, v, t) \in \mathcal{E}$ indicates an interaction from node u to node v at a specific timestamp $t \in \mathbb{R}$. Note that (u, v, t_i) and (u, v, t_j) are regarded as two different temporal edges if $t_i \neq t_j$. Namely, nodes u and v can be connected at multiple different timestamps. Let $n = |V|$ and $m = |\mathcal{E}|$ be the number of nodes and temporal edges, respectively. We use $t_{\max}(\mathcal{G})$ and $t_{\min}(\mathcal{G})$ to represent the maximum and minimum timestamp of $\mathcal{G}$, respectively. For any node subset $S \subseteq V$, we denote $\mathcal{G}_S(S, \mathcal{E}_S)$ is a temporal subgraph of $\mathcal{G}(V, \mathcal{E})$ iff $\mathcal{E}_S = \{(u, v, t) | u, v \in S\} \subseteq \mathcal{E}$. Note that compared with the previous work [8], [9], [44]–[48], the continuous-time temporal graph does not do any timestamp aggregation operation on the raw datasets. Thus, in this paper, we examine more general and purely temporal graphs that can preserve fine-grained dynamics in a lossless fashion [11]–[14]. For convenience, we use the temporal graph to represent the directed continuous-time temporal graph when the context is clear. We use $G(V, E)$ to denote the de-temporal graph of $\mathcal{G}$, in which $E = \{(u, v) | (u, v, t) \in \mathcal{E}\}$ and $|E| = \bar{m}$. That is, G is a static directed graph that ignores the timestamps of $\mathcal{G}$. Fig. 2(a) shows a de-temporal graph G of Fig. 1(a). Although there are many temporal motif definitions (Section VI), we use the most representative δ -temporal motif [18], [20], [21], [49], [50] due to its generality and usefulness, but our results can be easily extended to others [19]¹.

Definition 1: [δ -temporal motif [18]] A δ -temporal motif $\mathcal{M}_\delta$ is a temporal graph and $t_{\max}(\mathcal{M}_\delta) - t_{\min}(\mathcal{M}_\delta) \leq \delta$.

For understanding, we model each temporal graph as an edge stream [6], [48], which is a sequence of temporal edges incrementally ordered by timestamps. For example, the δ -temporal motif $\mathcal{M}_\delta$ can be represented as $\mathcal{M}_\delta = \{(x_1, y_1, t_1), (x_2, y_2, t_2), \dots, (x_l, y_l, t_l)\}$ where $t_i \leq t_{i+1}$ for all $1 \leq i \leq l-1$, and $t_l - t_1 \leq \delta$.

¹Since our models and algorithms only depend on temporal motif instances, other temporal motif definitions can be directly extended to our solutions by the corresponding enumeration or counting algorithms.

Definition 2: [δ -instance [18]] The temporal subgraph $\mathcal{G}_S = \{(u_1, v_1, t'_1), (u_2, v_2, t'_2), \dots, (u_l, v_l, t'_l)\}$, where $t'_i \leq t'_{i+1}$ for all $1 \leq i' \leq l-1$, is called a δ -instance of $\mathcal{M}_\delta$, denoted as $\mathcal{G}_S \in \mathcal{M}_\delta$, if the following conditions are satisfied.

- (i) **Temporal ordering:** There exists a bijection h on the nodes such that $h(x_i) = u_i$ and $h(y_i) = v_i$ for $i = 1, 2, \dots, l$;
- (ii) **Temporal duration:** The time differences of the temporal edges in $\mathcal{G}_S$ do not exceed δ , i.e., $t'_l - t'_1 \leq \delta$.

Fig. 2(b) and (c) show a toy δ -temporal motif and the corresponding δ -instances, respectively.

B. Problem Statement

By Definition 2, we can know that the time distribution of each δ -instance is different (Fig. 2(c)). Therefore, to differentiate the importance of different instances, we propose the following function that assigns a non-negative weight to each δ -instance, determined by its time distribution.

Definition 3: [δ -instance weighting function] Given a δ -instance $\mathcal{G}_S$, the weighting function of $\mathcal{G}_S$ is denoted by $f(\mathcal{G}_S) = \frac{1}{l-1} \sum_{i=2}^l g(-\beta(t_i - t_{i-1}))$, where $g(\cdot)$ is a decaying function that captures the time dependencies on adjacent interactions. For example, $g(x) = e^x$, which is often used in temporal settings [51], [52]. $0 \leq \beta \leq 1$ controls the time decay and a larger β accelerates its decay. Note that when $g(x) = e^x$ and $\beta = 0$, $f(\mathcal{G}_S) = 1$ is a constant function, which is used in our experiments.

Example 1: Reconsider the Fig. 2, there are two δ -instances $\mathcal{G}_{S_1} = \{(v_2, v_4), 1), (v_2, v_5), 4), (v_5, v_4), 5)\}$ and $\mathcal{G}_{S_2} = \{(v_3, v_6), 4), (v_3, v_4), 5), (v_4, v_6), 6)\}$. Let $\beta = 0.2$ and $g(x) = e^x$, we have $f(\mathcal{G}_{S_1}) = \frac{1}{2}(e^{-0.2*3} + e^{-0.2*1}) \approx 0.68$ and $f(\mathcal{G}_{S_2}) = \frac{1}{2}(e^{-0.2*1} + e^{-0.2*1}) \approx 0.819$. Thus, $\mathcal{G}_{S_2}$ has a greater weight than $\mathcal{G}_{S_1}$, which is intuitive because $\mathcal{G}_{S_2}$'s time distribution is more concentrated.

Motif-preserving graph clustering has been extensively studied in static networks, which aims to identify a cluster C such that the vertices within C participate in many motif instances and few motif instances cross C and $\bar{C} = V \setminus C$ [33]–[39]. However, existing methods ignore the frequently appearing temporal properties of graphs, resulting in sub-optimal performance (Section V). Inspired by this, we introduce a new temporal motif conductance, which generalizes the motif conductance [38] by integrating temporal information.

Definition 4: [temporal motif cut] Given a node subset C and a δ -temporal motif $\mathcal{M}_\delta$, the temporal motif cut of C is denoted by $cut_{\mathcal{M}_\delta}(C) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(S \cap C \neq \emptyset \& S \cap \bar{C} \neq \emptyset)$. Where $I(\cdot)$ is the indicator function.

Definition 5: [temporal motif degree] Given a δ -temporal motif $\mathcal{M}_\delta$ and a node u , the temporal motif degree of node u is denoted by $d_{\mathcal{M}_\delta}(u) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(u \in S)$.

Definition 6: [temporal motif volume] Given a node subset C and a δ -temporal motif $\mathcal{M}_\delta$, the temporal motif volume of C is denoted by $vol_{\mathcal{M}_\delta}(C) = \sum_{u \in C} d_{\mathcal{M}_\delta}(u)$.

Definition 7: [temporal motif conductance] Given a node set C and a δ -temporal motif $\mathcal{M}_\delta$, the temporal motif conductance of C is denoted by $\phi_{\mathcal{M}_\delta}(C) = \frac{cut_{\mathcal{M}_\delta}(C)}{\min\{vol_{\mathcal{M}_\delta}(C), vol_{\mathcal{M}_\delta}(\bar{C})\}}$.

TABLE I: A comprehensive comparison of the SOTA methods. Higher-order structures denote whether the objective function considers the informative motifs. Directed denotes whether the edges are directed or not. Note that *THDS* [53] only considers the internal higher-order cohesiveness and overlooks the critical external sparsity, which is an important factor in measuring the quality of a cluster [16], [29]–[31], resulting in unsatisfactory quality as stated in our empirical results. Our model *MgcCT* avoids this issue (details in Definition 7).

Methods	Continuous-time	Higher-order	Directed	Remark
<i>TMO</i> [9]	×	×	×	Modularity-based
<i>TSC</i> [8]	×	×	×	Conductance-based
<i>TDS</i> [11]	✓	×	×	Density-based
<i>HMO</i> [54], [55]	×	✓	✓	Modularity-based
<i>HSC</i> [37]–[39]	×	✓	✓	Conductance-based
<i>HDS</i> [56]–[58]	×	✓	✓	Density-based
<i>THDS</i> [53]	✓	✓	✓	Density-based
<i>MgcCT</i> (our)	✓	✓	✓	Conductance-based

Problem formulation. Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$, the goal of the Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*) is to identify a vertex subset C^* such that $\phi_{\mathcal{M}_\delta}(C^*) \leq \phi_{\mathcal{M}_\delta}(C)$ for any $C \subseteq V$. We use $\phi_{\mathcal{M}_\delta}^*$ to represent $\phi_{\mathcal{M}_\delta}(C^*)$.

According to Definition 7, we know that C^* is the sparsest balanced cluster, in which the sparsest signifies to minimize the δ -instances that cross C^* and $V \setminus C^*$, and the balanced indicates the δ -instances that are in the cluster. Thus, minimizing temporal motif conductance can find a cluster that preserves seamlessly higher-order temporal connectivity.

The advantages of *MgcCT*. (1) It provides a new clustering metric based on higher-order temporal motifs rather than simple temporal edges, allowing users to flexibly focus the clusters that highlight specific temporal motifs relevant to a given type of temporal network. For instance, temporal cycles are crucial features of financial markets, and emphasizing these clusters with dense temporal cycles can provide strong interpretability and improve the clustering quality. (2) The capability to handle directed temporal graphs for clustering is a largely advantage of our solutions, while existing methods are limited to undirected temporal graphs [8], [9], [11], [46]–[48]. For example, by the definition of δ -temporal motif (Definition 1), there are eight possible triangles on directed temporal graphs, but only one triangle exists on undirected temporal graphs, significantly missing this rich information. (3) It can be viewed as the minimum escape probability (Theorem 3.1) of a cluster, offering an alternative perspective based on random walks, which further reinforces its practical applicability. Table I reports the comprehensive comparison of the state-of-the-art (SOTA) graph clustering methods.

III. PROBLEM ANALYSIS AND EXISTING SOLUTIONS

A. Escape Probability and Problem NP-hardness

This subsection enhances our problem through the lens of escape probability. Then, we prove the problem's NP-hardness.

Definition 8: [cluster-constrained temporal motif transition matrix] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal

motif $\mathcal{M}_\delta$ with k nodes, and a cluster C . The cluster-constrained temporal motif transition matrix $\mathbf{P}_C \in R^{n \times n}$ on two nodes u and v with $(u, v) \in E$ can be computed as

$$\mathbf{P}_C(u \rightarrow v) = \begin{cases} \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{|C| d_{\mathcal{M}_\delta}(u)}, & v \notin C \\ 1 - \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{|C| d_{\mathcal{M}_\delta}(u)}, & v \in C \end{cases} \quad (1)$$

Implication of Definition 8: $\frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{|C| d_{\mathcal{M}_\delta}(u)}$ is positively correlated with $\frac{\sum_{i=1}^{k-1} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{k d_{\mathcal{M}_\delta}(u)}$.

Meanwhile, $\frac{\sum_{i=1}^{k-1} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{d_{\mathcal{M}_\delta}(u)}$ signifies the ratio of the temporal motif cut that includes node u to the temporal motif degree of node u . Therefore, when $v \notin C$, $\mathbf{P}_C(u \rightarrow v)$ is approximately equivalent to the random walk: u first selects a δ -instance that cuts C and $\bar{C}$ by their weight distribution, and then uniformly chooses a node of the δ -instance.

Theorem 3.1: [escape probability] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal motif $\mathcal{M}_\delta$ with k nodes, and a cluster C . Consider a random walk $(w_t)_{t \in \mathbb{N}}$ generated by the cluster-constrained temporal motif transition matrix $\mathbf{P}_C$, if the initial state w_0 is in C (or $\bar{C}$) and is randomly chosen following the distribution of temporal motif degree, we have

$$\phi_{\mathcal{M}_\delta}(C) = \max\{Pr(w_1 \in \bar{C} | w_0 \in C), Pr(w_1 \in C | w_0 \in \bar{C})\} \quad (2)$$

Proof: According to Definition 7, we have $\phi_{\mathcal{M}_\delta}(C) = \frac{cut_{\mathcal{M}_\delta}(C)}{\min\{vol_{\mathcal{M}_\delta}(C), vol_{\mathcal{M}_\delta}(\bar{C})\}} = \max\{\frac{cut_{\mathcal{M}_\delta}(C)}{vol_{\mathcal{M}_\delta}(C)}, \frac{cut_{\mathcal{M}_\delta}(\bar{C})}{vol_{\mathcal{M}_\delta}(\bar{C})}\}$. Then, according to Definitions 4-6, we have

$$\begin{aligned} \frac{cut_{\mathcal{M}_\delta}(C)}{vol_{\mathcal{M}_\delta}(C)} &= \frac{\sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(S \cap C \neq \emptyset \& S \cap \bar{C} \neq \emptyset)}{vol_{\mathcal{M}_\delta}(C)} \\ &= \frac{\sum_{u \in C} \sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{vol_{\mathcal{M}_\delta}(C)} \\ &= \sum_{u \in C} \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(|S \cap C| = i \& u \in S)}{d_{\mathcal{M}_\delta}(u)} \frac{d_{\mathcal{M}_\delta}(u)}{vol_{\mathcal{M}_\delta}(C)} \\ &= \sum_{u \in C} \sum_{v \in \bar{C}} \mathbf{P}_C(u \rightarrow v) \frac{d_{\mathcal{M}_\delta}(u)}{vol_{\mathcal{M}_\delta}(C)} \\ &= Pr(w_1 \in \bar{C} | w_0 \in C) \end{aligned}$$

Analogously, we have $\frac{cut_{\mathcal{M}_\delta}(\bar{C})}{vol_{\mathcal{M}_\delta}(\bar{C})} = Pr(w_1 \in C | w_0 \in \bar{C})$. Therefore, the theorem is proved. $\square$

Theorem 3.2: [Hardness] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$, the problem of identifying *MgcCT* from $\mathcal{G}(V, \mathcal{E})$ is NP-hard.

Proof: To prove that *MgcCT* is NP-hard, we reduce the minimum motif conductance problem to our *MgcCT* problem. Specifically, (1) *Instance Construction*: Let $G(V, E)$ be a de-temporal (i.e., static) graph and M be a de-temporal (i.e., static) motif. The minimum motif conductance problem is defined as finding a cluster $C \subseteq V$ that minimizes $\phi_M(C) =$

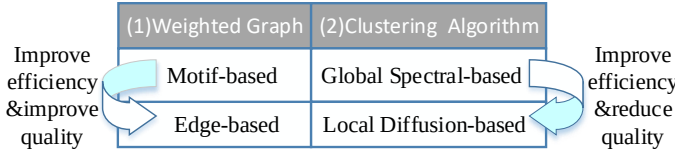


Fig. 3: Key functions of the seminal two-stage framework.

$\frac{cut_M(C)}{\min(vol_M(C), vol_M(\bar{C}))}$, where $cut_M(C)$ denotes the number of M -instances crossing the boundary between C and $\bar{C}$, and $vol_M(C)$ denotes the total number of M -instances involving nodes in C . (2) *Reduction to MgcCT*: Construct a temporal graph $\mathcal{G}(V, \mathcal{E})$ from $G(V, E)$ by assigning all edges the same timestamp t_0 (i.e., $t_{max}(\mathcal{G}) = t_{min}(\mathcal{G}) = t_0$). Define a δ -temporal motif $\mathcal{M}_\delta$ as the temporal extension of M with $\delta = 0$, ensuring that $\mathcal{M}_\delta$ -instances in $\mathcal{G}$ are exactly equivalent to M -instances in G . (3) *Equivalence Verification*: For any cluster $C \subseteq V$, the temporal motif conductance of C in $\mathcal{G}$ (Definition 7) satisfies: $\phi_{\mathcal{M}_\delta}(C) = \frac{cut_{\mathcal{M}_\delta}(C)}{\min(vol_{\mathcal{M}_\delta}(C), vol_{\mathcal{M}_\delta}(\bar{C}))} = \phi_M(C)$. Thus, finding the cluster C^* that minimizes $\phi_{\mathcal{M}_\delta}(C)$ (the *MgcCT* problem) is equivalent to finding the cluster C^* that minimizes $\phi_M(C)$ (the minimum motif conductance problem). (4) *Conclusion*: Since the minimum motif conductance problem is NP-hard [38], and the above reduction can be completed in polynomial time, the *MgcCT* problem is also NP-hard. $\square$

B. Existing Solutions and Their Limitations

According to Theorem 3.2, the intrinsic complexity of the proposed problem necessitates the development of approximate algorithms to efficiently optimize our objectives. At a high level, most existing methods adhere to the well-known two-stage clustering framework [28], [36]–[38]. As shown in Figure 3, in the first stage, the input graph is transformed into a static weighted graph, where the weight of each edge corresponds to the number of motif instances in which the edge participates. In the second stage, the static weighted graph is partitioned using any graph clustering algorithm. Different algorithms differentiate themselves in how they efficiently and effectively partition the static weighted graph. For instance, Benson et al. [38] and Yin et al. [36] employ the Fiedler vector-based global spectral clustering algorithm and the local diffusion graph clustering algorithm, respectively, to partition the static weighted graph. Building on this idea, we introduce the following key concepts. Let $\mathcal{A} \in \mathbb{R}^{n \times n}$ denote the δ -temporal motif weighted graph (matrix), where $A_{uv} = A_{vu} = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(u \in S \wedge v \in S)$ for all $u, v \in V$. It is crucial to note that u and v may not necessarily be connected by an edge in the original de-temporal graph (i.e., $(u, v) \notin E$). Using these notations, several basic existing solutions are outlined below (details in their original papers).

1) *Order-Based Solutions*: These solutions can be summarized in two steps: (1) Sort all vertices according to a predefined rule. Without loss of generality, let the sorted vertices be denoted as $u_1, u_2, \dots, u_n$. (2) Output $S = \arg \min \phi_{\mathcal{A}}(S_i)$, where $S_i = \{u_1, u_2, \dots, u_i\}$ and $\phi_{\mathcal{A}}(S_i)$ represents the traditional edge-based conductance [60], [61] of S_i on the weighted

graph $\mathcal{A}$. Specifically, $\phi_{\mathcal{A}}(S_i) = \frac{\sum_{u \in S_i, v \notin S_i} A_{uv}}{\min\{\sum_{u \in S_i} D_{uu}, \sum_{v \notin S_i} D_{vv}\}}$, where $D_{uu} = \sum_v A_{uv}$. Consequently, the following two order-based solutions are proposed: *THSC* [38] and *THLC* [36]. In particular, *THSC* [38] and *THLC* [36] utilize the Fiedler Vector and Personalized PageRank vector, respectively, to sort all vertices. Note that [62] and *THLC* [36] are conceptually similar: both approaches begin by enumerating temporal motif instances and subsequently perform local graph diffusion. The main distinction is that [62] implements the diffusion process via the tensor formed by all motif instances. Therefore, [62] remains largely of theoretical interest, as it is challenging to implement in practice and suffers from poor and highly unstable empirical performance, largely due to the presence of numerous hard-to-tune parameters [27], [28]. Given that *THLC* [36] has already demonstrated superior empirical results over [62], we omit further experimental comparisons, as they would be redundant. For convenience, Table II summarizes these basic solutions for comparison.

Limitations of Order-Based Solutions. From Table II, we observe the following: (1) Although *THSC* has successfully established the Cheeger inequality (i.e., quadratic bound) for motifs with three vertices, its applicability to motifs containing four or more vertices remains unproven. This limitation is significant because larger motifs are often more practical and widely used in various applications [19], [41]–[43]. (2) *THLC* is a heuristic algorithm, and its clustering quality is significantly influenced by numerous hard-to-tune parameters and seeding strategies. In practical applications, its performance can be unstable and may even yield very poor solutions, as demonstrated by our experimental results. (3) *THSC* and *THLC* suffer from significant computational and memory inefficiencies, primarily due to their need to enumerate and store all temporal motif instances.

2) *The Peeling-based Solution*: Recently, Lin et al. [28] presented a peeling-based method that seamlessly integrates the computation and partitioning of $\mathcal{A}$ in an iterative fashion, yielding results that come with approximation guarantees. Specifically, they redefine the concept of motif conductance, where temporal motif conductance represents a specialized instance when the motif is a δ -temporal motif, and introduce a peeling-based greedy algorithm that iteratively refines motif conductance by initiating the optimization process from each individual vertex. To align with the specific requirements of our problem, we provide the following definitions.

Definition 9: (Temporal Motif Resident) For a vertex set C , the temporal motif resident of a vertex $u \in C$ with respect to C is defined as: $Tr_C(u) = \frac{d_{\mathcal{M}_\delta}(u) + d_{\mathcal{M}_\delta}(u, C) - d_{\mathcal{M}_\delta}(u, \bar{C})}{d_{\mathcal{M}_\delta}(u)}$, in which $d_{\mathcal{M}_\delta}(u, H) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(u \in S, S - \{u\} \subseteq H)$ and H can be either C or $\bar{C}$.

By Definition 9, the Temporal Higher-order Peeling Removal (*THPR*) algorithm is outlined as follows: First, it calculates the temporal motif resident for each vertex. Subsequently, it iteratively eliminates the vertex with the smallest temporal motif. Finally, it outputs the result that exhibits the smallest

TABLE II: A comparison of *MgcCT* algorithms. k is the number of nodes of the given temporal motif. Note that $k = 2$ is not discussed, because in this situation, our model degenerates into the traditional lower-order temporal graph clustering and not the focus of this paper. $\times$ represents the corresponding method has no accuracy guarantee. $\phi_{\mathcal{M}_\delta}^* \in (0, 1]$ is the smallest temporal motif conductance. $O(E(\mathcal{M}_\delta))$ (resp., $O(C(\mathcal{M}_\delta))$) is the time complexity of enumeration (resp., counting) temporal motif instances. $num(\mathcal{M}_\delta)$ is the number of temporal motif instances. $\Delta(u)$ is the time cost of calculating the temporal motif degree of node u and $\Delta_{max} = \max\{\Delta(u) | u \in V\}$. α and ϵ are parameters less than 1. $O(E(\mathcal{M}_\delta))$ is much slower than $O(C(\mathcal{M}_\delta))$ [18], [41]–[43], [59].

Methods	Accuracy Guarantee	Time Complexity	Space Complexity	Weighted Graph
<i>THSC</i> [38]	$O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$ for $k = 3$; $\times$ for $k > 3$	$O(E(\mathcal{M}_\delta))$	$O(\min\{n^2, \binom{k}{2} num(\mathcal{M}_\delta)\})$	Motif-based
<i>THLC</i> [36]	$\times$	$O(E(\mathcal{M}_\delta))$	$O(\min\{n^2, \binom{k}{2} num(\mathcal{M}_\delta)\})$	Motif-based
<i>THPR</i> [28]	$O(1/2 + 1/2\phi_{\mathcal{M}_\delta}^*)$ for any k	$O(E(\mathcal{M}_\delta))$	$O(m)$	Motif-based
<i>AGA</i> (Our)	$O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$ for any k	$O(C(\mathcal{M}_\delta))$	$O(m)$	Edge-based
<i>HLA</i> (Our)	$\times$	$O(\frac{\Delta_{max}}{\alpha\epsilon})$	$O(\frac{1}{\alpha\epsilon})$	Edge-based

temporal motif conductance during the iteration procedure.

Limitations of the Peeling-Based Approach. As indicated in Table II, although *THPR* offers results with approximation guarantees, our empirical evaluation reveals that it exhibits suboptimal practical clustering quality. More importantly, *THPR* imposes excessively high time costs due to the necessity of enumerating all temporal motif instances, which leads to a combinatorial explosion and diminished efficiency.

IV. THE PROPOSED SOLUTIONS FOR *MgcCT*

A. Novel Edge-focused Weighted Graph Projection

By Table II, it is evident that *THSC* is theoretically well-justified when $k = 3$. Nevertheless, in the more general scenario where $k > 3$, *THSC* fails to deliver results accompanied by provable approximation guarantees. The primary impediment lies in the ambiguity surrounding the correlation between the temporal motif cut $cut_{\mathcal{M}_\delta}(C)$ and the weighted graph cut $\sum_{u \in C, v \notin C} \mathcal{A}_{uv}$, a topic extensively discussed in [38]. Specifically, the question of whether $cut_{\mathcal{M}_\delta}(C)$ can be constrained by $\sum_{u \in C, v \notin C} \mathcal{A}_{uv}$ remains unresolved [38]. The underlying rationale for this limitation is that the existing reweighting clustering approach initially constructs a hypergraph with hyperedges representing all temporal motif instances. Subsequently, it converts this hypergraph into a weighted graph, wherein the weight of an edge (u, v) is indicative of the number of times the vertex pair $\{u, v\}$ is involved in the hyperedges. This transformation process, however, leads to the forfeiture of the original edge information. Consequently, a pivotal inquiry arises: Can we devise a novel weighted graph projection method that facilitates the analysis of the relationship between $cut_{\mathcal{M}_\delta}(C)$ and $\sum_{u \in C, v \notin C} \mathcal{A}_{uv}$, while simultaneously allowing for the examination of the association between the temporal motif volume $vol_{\mathcal{M}_\delta}(C)$ and weighted graph volume $\sum_{u \in C} \mathcal{D}_{uu}$? In this subsection, we offer an affirmative response to this question by introducing an innovative edge-focused weighted graph projection method that retains the original edge information.

Definition 10: [edge-focused weighted matrix] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$, the edge-focused weighted matrix $W_{ef} \in R^{n \times n}$ is as follows:

$$(W_{ef})_{uv} = \begin{cases} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I((u, v) \in \mathcal{G}_S), & (u, v) \in G \\ 0, & (u, v) \notin G \end{cases} \quad (3)$$

Remark. By Definition 10, it is apparent that the distinctive feature of our edge-focused weighted matrix (graph) in comparison to existing motif-based weighted graphs is that it exclusively assigns weights to edges that are present in the original graph. Specifically, if an edge (u, v) does not exist in E , its edge-focused weight is set to zero. In essence, an edge in the edge-focused weighted graph (matrix) possesses a non-zero weight if and only if it also appears as an edge in the original bipartite graph G . We underscore that, despite its simplicity, this modification engenders several substantial benefits: (1) We can achieve high-quality clustering results with provable approximation guarantees (Theorem 4.3), a novel theoretical advancement not found in existing literature. (2) In the worst-case scenario, the size of the edge-focused weighted graph W_{ef} is $O(\tilde{m})$, which is more compact than the motif-based weighted graph $\mathcal{A}$, whose size is $O(\min\{n^2, \binom{k}{2} num(\mathcal{M}_\delta)\})$, where $num(\mathcal{M}_\delta)$ represents the number of motif instances. Consequently, partitioning W_{ef} requires less time and space compared to $\mathcal{A}$. (3) The edge-focused weighted matrix can be directly derived using combinatorial counting techniques, eliminating the need for exhaustive enumeration of all temporal motif instances, thereby significantly reducing computational overhead.

The following pivotal lemmas that illuminate the correlation between $cut_{\mathcal{M}_\delta}(\cdot)$ and $vol_{\mathcal{M}_\delta}(\cdot)$ within the original temporal graph $\mathcal{G}$ and our proposed edge-focused weighted graph W_{ef} , which establish a solid theoretical foundation that underpins the development of our subsequent algorithms.

Lemma 4.1: Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a cluster $C \subseteq V$, and a δ -temporal motif $\mathcal{M}_\delta$ with m_δ de-temporal edges, we have $cut_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C, v \notin C} (W_{ef})_{uv} \leq m_\delta cut_{\mathcal{M}_\delta}(C)$.

Proof: Let $[C, \bar{C}] = \{(u, v) \in G | u \in C, v \in \bar{C}\}$. According to Definition 10, we have $\sum_{u \in C, v \notin C} (W_{ef})_{uv}$

$$\begin{aligned}
&= \sum_{(u,v) \in [C, \bar{C}]} (W_{ef})_{uv} = \sum_{(u,v) \in [C, \bar{C}]} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I((u,v) \in \mathcal{G}_S) \\
&= \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) \sum_{(u,v) \in [C, \bar{C}]} I((u,v) \in \mathcal{G}_S) \\
&= \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) |[C, \bar{C}] \cap \mathcal{G}_S|, \text{ in which } \mathcal{G}_S \text{ is the} \\
&\text{de-temporal graph of } \mathcal{G}_S. \text{ For any } \mathcal{G}_S \in \mathcal{M}_\delta \text{ and } \\
&|[C, \bar{C}] \cap \mathcal{G}_S| \neq \emptyset, \text{ we have } 1 \leq |[C, \bar{C}] \cap \mathcal{G}_S| \leq m_\delta. \\
&\text{Thus, } \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I([C, \bar{C}] \cap \mathcal{G}_S) \leq \sum_{u \in C, v \notin C} (W_{ef})_{uv} \leq \\
&m_\delta \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I([C, \bar{C}] \cap \mathcal{G}_S). \text{ On the other hand, by} \\
&\text{Definition 4, we have } \text{cut}_{\mathcal{M}_\delta}(C) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(S \cap C \neq \\
&\emptyset \& S \cap \bar{C} \neq \emptyset) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I([C, \bar{C}] \cap \mathcal{G}_S). \text{ Therefore,} \\
&\text{we have } \text{cut}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C, v \notin C} (W_{ef})_{uv} \leq m_\delta \text{cut}_{\mathcal{M}_\delta}(C). \quad \square
\end{aligned}$$

Lemma 4.2: Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a cluster $C \subseteq V$, and a δ -temporal motif $\mathcal{M}_\delta$ with m_δ de-temporal edges, we have $\text{vol}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C} \sum_{v \in V} (W_{ef})_{uv} \leq m_\delta \text{vol}_{\mathcal{M}_\delta}(C)$.

Proof: According to Definition 10, for any $u \in C$, we have $\sum_{v \in V} (W_{ef})_{uv} = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} \sum_{(u,v) \in \mathcal{G}_S} f(\mathcal{G}_S) I((u,v) \in \mathcal{G}_S)$, in which $\mathcal{G}_S$ is the de-temporal graph of $\mathcal{G}_S$. Meanwhile, according to Definition 6, we have $\text{vol}_{\mathcal{M}_\delta}(C) = \sum_{u \in C} d_{\mathcal{G}}^{\mathcal{M}_\delta}(u) = \sum_{u \in C} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(u \in \mathcal{G}_S)$. The following cases are stated.

Case 1: For any $\mathcal{G}_S \in \mathcal{M}_\delta$ and $u \notin \mathcal{G}_S$, $f(\mathcal{G}_S) I(u \in \mathcal{G}_S) = 0$ and $\sum_{(u,v) \in \mathcal{G}_S} f(\mathcal{G}_S) I((u,v) \in \mathcal{G}_S) = 0$.

Case 2: For any $\mathcal{G}_S \in \mathcal{M}_\delta$ and $u \in \mathcal{G}_S$, $f(\mathcal{G}_S) \leq \sum_{(u,v) \in \mathcal{G}_S} f(\mathcal{G}_S) I((u,v) \in \mathcal{G}_S) \leq m_\delta f(\mathcal{G}_S)$.

Thus, we have $\sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(u \in \mathcal{G}_S) \leq \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} \sum_{(u,v) \in \mathcal{G}_S} f(\mathcal{G}_S) I((u,v) \in \mathcal{G}_S) \leq m_\delta \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} f(\mathcal{G}_S) I(u \in \mathcal{G}_S)$. Furthermore, $\text{vol}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C} \sum_{v \in V} (W_{ef})_{uv} \leq m_\delta \text{vol}_{\mathcal{M}_\delta}(C)$. $\square$

Implications of Lemma 4.1 and Lemma 4.2. Drawing upon Lemma 4.1 and Lemma 4.2, it becomes evident that the proposed edge-focused weighted graph W_{ef} can approximate both $\text{vol}_{\mathcal{M}_\delta}(\cdot)$ and $\text{cut}_{\mathcal{M}_\delta}(\cdot)$ of the original temporal graph. As a result, when spectral clustering is applied to W_{ef} , it offers provable approximation guarantees (details in Theorem 4.3). However, existing motif-based methods have inaccurate estimates for them, resulting in subpar quality (Table II).

B. AGA: The Approximate Global Algorithm

Based on the preceding analyses, we propose an approximate global algorithm, denoted as AGA, which is detailed in Algorithm 1. Specifically, Algorithm 1 initiates by computing the edge-focused weighted matrix W_{ef} (Lines 1-4). Subsequently, the algorithm derives the normalized Laplacian matrix, $\mathcal{L}$, utilizing the previously obtained W_{ef} (Line 7). Following this, Algorithm 1 proceeds to identify the eigenvector z that corresponds to the second smallest eigenvalue of $\mathcal{L}$, facilitating the execution of the sweep-cut procedure (Lines 8-10). This process entails arranging all entries of z in descending order, ensuring $z_1 \geq z_2 \geq \dots \geq z_n$,

Algorithm 1 Approximate Global Algorithm (AGA)

Input: A temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$
Output: A cluster C

- 1: $G \leftarrow$ the de-temporal graph of $\mathcal{G}$
- 2: Initializing W_{ef} and $\mathcal{D}$ are zero matrices
- 3: **for** each edge $(u, v) \in G$ **do**
- 4: $(W_{ef})_{uv} \leftarrow$ the edge-focused weight by Eq. 3
- 5: $(W_{ef})_{vu} \leftarrow (W_{ef})_{uv}$
- 6: $\mathcal{D}_{uu} += (W_{ef})_{uv}; \mathcal{D}_{vv} += (W_{ef})_{uv}$
- 7: $\mathcal{L} \leftarrow I - \mathcal{D}^{-1/2} W_{ef} \mathcal{D}^{-1/2}$
- 8: $z \leftarrow$ eigenvector of the second smallest eigenvalue of $\mathcal{L}$
- 9: $z_i \leftarrow$ to be index of z with i th smallest value
- 10: $C \leftarrow \arg \min \phi_{W_{ef}}(C_i)$, where $C_i = \{z_1, z_2, \dots, z_i\}$ and $\phi_{W_{ef}}(C_i)$ is the edge-based conductance of C_i in W_{ef}
- 11: **if** $|C| > |\bar{C}|$ **then**
- 12: **return** $\bar{C}$
- 13: **else**
- 14: **return** C

and then selecting the set $C = \arg \min \phi_{W_{ef}}(C_i)$, where $C_i = \{z_1, z_2, \dots, z_i\}$ and $\phi_{W_{ef}}(C_i)$ denotes the conventional edge-based conductance [60], [61] of C_i on W_{ef} . Namely, $\phi_{W_{ef}}(C_i) = \frac{\sum_{u \in C_i, v \notin C_i} (W_{ef})_{uv}}{\min\{\sum_{u \in C_i} \sum_{v \in V} (W_{ef})_{uv}, \sum_{u \notin C_i} \sum_{v \in V} (W_{ef})_{uv}\}}$. Finally, Algorithm 1 outputs the smaller set between C and $\bar{C}$ (Lines 11-14), given that $\phi_{W_{ef}}(C) = \phi_{W_{ef}}(\bar{C})$.

Theoretical Analysis. Next, we analyze the time&space complexities and clustering quality of Algorithm 1.

Theorem 4.1: Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$, the time and space complexity of Algorithm 1 are $O(C(\mathcal{M}_\delta))$ and $O(m)$, respectively. $O(C(\mathcal{M}_\delta))$ is the time complexity of counting temporal motif instances.

Proof: Initially, Algorithm 1 requires $O(C(\mathcal{M}_\delta))$ time to construct the edge-focused weighted graph and its corresponding diagonal degree matrix (Lines 2-6). Following this, in Line 7, the algorithm utilizes $O(\bar{m})$ time to derive the Laplacian matrix $\mathcal{L}$, given that the edge-focused weighted graph's size is bounded by $O(\bar{m})$ (as per Definition 10). Subsequently, in Line 8, Algorithm 1 employs the well-established Lanczos iteration algorithm [63] to compute the eigenvector associated with the second smallest eigenvalue of $\mathcal{L}$, which also takes $O(\bar{m})$ time. Finally, the algorithm dedicates $O(n \log n)$ time to sort the indices (Line 9) and $O(\bar{m})$ time to calculate the conductance of S_i on the edge-focused weighted graph (Line 10). Consequently, the overall time complexity of Algorithm 1 amounts to $O(C(\mathcal{M}_\delta))$ due to $C(\mathcal{M}_\delta) > \bar{m} > n \log n$.

For space complexity, Algorithm 1 needs an additional $O(\bar{m})$ space to store W_{ef} , $\mathcal{D}$, and $\mathcal{L}$. Additionally, it requires $O(m)$ space to store the input graph G . Hence, the total space complexity of Algorithm 1 is $O(m)$ due to $m > \bar{m}$. $\square$

Remark. By Table II, it is clear that existing methods depend on temporal motif enumeration algorithms to obtain the hyperedges (i.e., temporal motif instances) and then construct the motif-based weighted graph for clustering. In stark contrast, our novel AGA algorithm is fundamentally based on the temporal motif counting method, which can significantly reduce the time complexity. Furthermore, as stated in Theorem 4.1, AGA necessitates only linear space complexity for the

storage of basic data structures. This is unlike other algorithms (except for *THPR* [28]), which typically need to store all temporal motif instances, resulting highly space complexity. Consequently, our proposed *AGA* algorithm surpasses SOTA methods in terms of efficiency, as stated in our experiments.

Theorem 4.2: Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$ with m_δ de-temporal edges, we let λ_2 be the second smallest eigenvalue of the normalized Laplacian matrix $\mathcal{L}$ (Line 7). We have

$$\frac{\lambda_2}{2m_\delta} \leq \phi_{\mathcal{M}_\delta}^* \leq m_\delta \sqrt{2\lambda_2} \quad (4)$$

Proof: For an arbitrary cluster C , leveraging Lemma 4.1 and Lemma 4.2, we establish the following inequalities: $\text{vol}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C} \sum_{v \in V} (W_{ef})_{uv} \leq m_\delta \text{vol}_{\mathcal{M}_\delta}(C)$ and $\text{cut}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C, v \notin C} (W_{ef})_{uv} \leq m_\delta \text{cut}_{\mathcal{M}_\delta}(C)$. Consequently, we derive the relationship $\frac{\phi_{\mathcal{M}_\delta}(C)}{m_\delta} \leq \phi_{W_{ef}}(C) \leq m_\delta \phi_{\mathcal{M}_\delta}(C)$, where $\phi_{W_{ef}}(C)$ denotes the conventional edge-based conductance [60], [61] of C on the edge-focused weighted graph W_{ef} . Furthermore, invoking the Cheeger inequality [64], we obtain $\lambda_2/2 \leq \phi_{W_{ef}}^* \leq \sqrt{2\lambda_2}$, where $\phi_{W_{ef}}^*$ represents the smallest conductance among all $\phi_{W_{ef}}(C)$. Therefore, we conclude that $\frac{\lambda_2}{2m_\delta} \leq \phi_{\mathcal{M}_\delta}^* \leq m_\delta \sqrt{2\lambda_2}$, thereby completing the proof of the theorem. $\square$

Theorem 4.3: Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$ with m_δ de-temporal edges, we let C be the cluster outputted by Algorithm 1. We have

$$\phi_{\mathcal{M}_\delta}(C) = O(\sqrt{\phi_{\mathcal{M}_\delta}^*}) \quad (5)$$

Proof: During the sweep-cut operation executed in Lines 7-10, we observe that $\phi_{W_{ef}}(C) \leq \sqrt{2\lambda_2}$, a consequence stemming from the inherent properties of the Fiedler vector ([61], [65]). Consequently, leveraging the proof methodology outlined in Theorem 4.2, we establish that $\frac{\phi_{\mathcal{M}_\delta}(C)}{m_\delta} \leq \phi_{W_{ef}}(C) \leq \sqrt{2\lambda_2}$. Furthermore, the Cheeger inequality [64] dictates that $\lambda_2/2 \leq \phi_{W_{ef}}^*$. By integrating this insight into the proof framework of Theorem 4.2, we derive that $\frac{\phi_{\mathcal{M}_\delta}(C)}{m_\delta} \leq \sqrt{2\lambda_2} \leq 2\sqrt{\phi_{W_{ef}}^*} \leq 2\sqrt{m_\delta \phi_{\mathcal{M}_\delta}^*}$. Therefore, we have $\phi_{\mathcal{M}_\delta}(C) \leq 2m_\delta \sqrt{m_\delta \phi_{\mathcal{M}_\delta}^*} = O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$, which completes the proof of the theorem. $\square$

Remark. According to Table II, our proposed algorithm *AGA* attains an asymptotically equivalent level of clustering quality when compared to the *THSC* method. Notably, while *THSC* cannot furnish quality assurances when $k > 3$, our *AGA* guarantees uniform quality performance across all values of k . In contrast, evaluating the clustering quality of *AGA* against *THPR* presents a more intricate analysis, as the performance is contingent upon $\phi_{\mathcal{M}_\delta}^*$. Nevertheless, as evidenced by the empirical findings outlined in Section V, *AGA* consistently outperforms *THPR*, a phenomenon that can be attributed to the fact that *THPR* is designed with a generic graph focus, thereby neglecting the unique temporal dynamics inherent in temporal graphs. Thus, our proposed *AGA* method yields clusters that are not only theoretically grounded in high-quality guarantees but also empirically validated to surpass SOTA approaches.

TABLE III: The ratio of the number of vertices in the identified cluster to that of the whole graph.

Dataset	# vertices	M_1	M_2	M_3
CM	1,893	46(2.43%)	15(0.79%)	95(5.01%)
UB	152,599	12(0.0078%)	55(0.036%)	14(0.0091%)
SU	189,191	108(0.057%)	403(0.21%)	6(0.0032%)
SO	2,572,345	41(0.0015%)	14(0.00054%)	49(0.0019%)

C. HLA: The Heuristic Local Algorithm

Although our global approximate algorithm, *AGA*, boasts a lower time complexity than existing methods, it still falls short in efficiency when handling large temporal graphs. For instance, on the SO dataset with nearly 50 million temporal edges, *AGA* requires about 3 hours to process a single query (Section V). This delay significantly disrupts the online user experience. The underlying reasons for this inefficiency can be attributed as follows: *AGA* must precompute the weight $(W_{ef})_{uv}$ for every edge (u, v) (Lines 3-6 of Algorithm 1), a process that consumes the majority of its execution time (Theorem 4.1). Fortunately, on most real-world graphs, the target clusters are often localized, meaning that the proportion of the vertices belonging to the target cluster is very small [27], [28], [36], [38]. As shown in Table III, the ratio of the number of vertices in the target cluster comprises no more than 5% of the total vertices in the original graph. Besides, in numerous real-world scenarios, users are often willing to tolerate a certain degree of inaccuracy in exchange for faster response times, especially in large networks. As a result, it is desirable to develop heuristic local solutions that can strike a better balance between efficiency and quality.

Inspired by the well-known local higher-order graph clustering [36], we introduce a heuristic *pure* local algorithm *HLA* to address the above challenges. Note that the proposed *HLA* is built upon [36], but incorporates more novel strategies to reduce computational overheads. In particular, [36] is not a purely local algorithm because it still requires enumerating motif instances to construct a weighted graph as an index. In contrast, our objective is to develop a purely local algorithm that operates by accessing only a small fraction of the graph, without introducing any additional overhead associated with indexing. Before proceeding further, we briefly review the simple but efficient subroutine *Push* [66], which is used to approximate the well-known Personalized PageRank vector. The *Push* algorithm initiates its operation from a source state (node) s and proceeds to disseminate information locally. Throughout its execution, the algorithm iteratively refines two key variables associated with each state u : its reserve $\pi(u)$ and its residue $r(u)$. Specifically, $\pi(u)$ serves as an approximation of the Personalized PageRank value of state u with respect to the source state s , while $r(u)$ quantifies the information that remains to be propagated from state u to other states. In each iteration (Lines 22-27 of Algorithm 2), the *Push* algorithm identifies states u that require information dissemination (Line 23). For each such state u , the algorithm transfers a portion $\alpha r(u)$ to the reserve $\pi(u)$ (Line 27), while the remaining

Algorithm 2 *Heuristic Local Algorithm (HLA)*

Input: A temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal motif $\mathcal{M}_\delta$, a source node s and two parameters α and ϵ

Output: A cluster C

```

1:  $r(s) \leftarrow 1$ ;  $r(u) \leftarrow 0$  for all  $u \neq s$ ;  $\pi(u) \leftarrow 0$  for all  $u \in V$ 
2:  $Q \leftarrow \{s\}$ ;  $D \leftarrow \{\}$ ;  $G \leftarrow$  the de-temporal graph of  $\mathcal{G}$ 
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow Q.pop()$ 
5:   if  $u \notin D$  then
6:      $D \leftarrow D \cup \{u\}$ 
7:      $TMDEGREE(u)$ 
8:    $PUSH(u)$ 
9: for  $u \in D$  do
10:   $\pi(u) \leftarrow \pi(u)/d_{\mathcal{M}_\delta}(u)$ 
11:  $\pi_i \leftarrow$  to be index of  $\pi$  with  $i$ th largest non-zero value
12:  $C \leftarrow \arg \min \phi_{\mathcal{M}_\delta}(C_i)$ , where  $C_i = \{\pi_1, \pi_2, \dots, \pi_i\}$ 
13: if  $|C| > |\bar{C}|$  then
14:   return  $C$ 
15: else
16:   return  $C$ 
17: function  $TMDEGREE(u)$ 
18:   $d_{\mathcal{M}_\delta}(u) \leftarrow 0$ 
19:  for each edge  $(u, v) \in G$  do
20:     $(W_{ef})_{uv} \leftarrow$  the edge-focused weight by Eq. 3
21:     $d_{\mathcal{M}_\delta}(u) += (W_{ef})_{uv}$ 
22: function  $PUSH(u)$ 
23:  if  $r(u) \geq \epsilon \cdot d_{\mathcal{M}_\delta}(u)$  then
24:    for each  $v \in \{w | (W_{ef})_{uw} \neq 0\}$  do
25:       $r(v) \leftarrow r(v) + (1 - \alpha)r(u) \cdot (W_{ef})_{uv}/d_{\mathcal{M}_\delta}(u)$ 
26:       $Q.push(v)$ 
27:   $\pi(u) \leftarrow \pi(u) + \alpha r(u)$ ,  $r(u) \leftarrow 0$ 

```

$(1 - \alpha)r(u)$ is distributed among the neighboring states in proportion to the edge weights connecting them, rather than being evenly distributed (Lines 24-25). After finishing the propagation, the algorithm resets the residue $r(u)$ to zero (Line 27). Following [36], [66], ϵ is a parameter used to balance quality and efficiency.

Based on these backgrounds, Algorithm 2 first initializes the residue $r(s) = 1$ for the source node s and 0 for all other nodes, the reserve $\pi(u) = 0$ for all $u \in V$, $Q = \{s\}$ as the nodes to be pushed, $D = \{\}$ as the nodes whose weight information has already been calculated to avoid repeated calculation (Lines 1-2). In Lines 3-8, it locally calculates temporal motif degree and edge weights on demand to perform the push operation (Line 8, Lines 22-27) until the queue Q becomes empty (Line 3). In particular, Line 4 pops a vertex u from the queue Q and then determines whether the temporal motif degree of u has been calculated. If not (Line 5), it adds u into set D (Line 6) and executes Line 7 to obtain the temporal motif degree of u and the edge-focused weight of its neighbor (Lines 17-21). After the propagation, it updates the reserve $\pi(u)$ for all $u \in D$ through temporal motif degree normalization (Lines 9-10). Finally, similar to Lines 9-14 of Algorithm 1, Algorithm 2 also performs the sweep-cut operation to obtain the target cluster C (Lines 11-16). Note that Line 11 only sorts all non-zero entries in π such that $\pi_1 \geq \pi_2 \geq \dots \geq \pi_{sup(\pi)}$ ($sup(\pi)$ is the number of the non-zero entries in π and $sup(\pi) \ll n$ [66]), which further guarantees the locality of Algorithm 2.

Theorem 4.4: Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal

TABLE IV: Dataset statistics.

Dataset	$n = V $	$m = \mathcal{E} $	$\bar{m} = E $	Time Span
CM	1,893	59,794	20,292	193 days
EE	986	327,336	24,929	903 days
MO	24,668	390,348	227,933	2,350 days
SA	30,278	445,390	56,115	338 days
UB	152,599	723,628	541,751	2,613 days
FW	43,953	851,913	262,631	1561 days
SU	189,191	1,106,530	852,335	2,773 days
SO	2,572,345	47,895,496	34,868,599	2,774 days

motif $\mathcal{M}_\delta$, and two parameters α and ϵ , the time complexity and space complexity of Algorithm 2 are $O(\frac{\Delta_{max}}{\alpha\epsilon})$ and $O(\frac{1}{\alpha\epsilon})$, respectively. Where $\Delta(u)$ is the time cost of calculating the temporal motif degree of node u and $\Delta_{max} = \max\{\Delta(u) | u \in V\}$.

Proof: The time cost of Algorithm 2 is mainly consumed in calculating the temporal motif degree of nodes (Line 7) and performing push operations on nodes (Line 8). Let u_i be the node to be pushed in the i th push operation and n_i be the number of nodes that receive propagation during the i -th push operation is performed. When the i th push operation is performed (i.e., $r(u) \geq \epsilon \cdot d_{\mathcal{M}_\delta}(u)$ in Line 23), the residue $r(u_i) \geq \epsilon n_i$ due to $d_{\mathcal{M}_\delta}(u_i) \geq n_i$, so $\|r\|_1$ decreased by at least $\alpha\epsilon n_i$ (Lines 24-27). Since $\|r\|_1 = 1$ initially (Line 1) and $\|r\|_1$ is at most 1 throughout the entire push process because r always decreases (Lines 25 and 27), we have $\alpha\epsilon \sum_{i=1} n_i \leq 1$ and $\sum_{i=1} n_i \leq \frac{1}{\alpha\epsilon}$. Therefore, the time complexity of all push operations in Algorithm 2 is $O(\frac{1}{\alpha\epsilon})$. On the other hand, we only calculate the temporal motif degree $d_{\mathcal{M}_\delta}(u)$ for vertex u in the queue Q instead of all nodes in advance. Thus, by Line 26, a total of $O(\frac{1}{\alpha\epsilon})$ vertices have ever been in Q . As a result, the time complexity of all calculations of the temporal motif degree in Algorithm 2 is $O(\frac{\Delta_{max}}{\alpha\epsilon})$. So, the overall time complexity of Algorithm 2 is $O(\frac{\Delta_{max}}{\alpha\epsilon})$. Regarding space complexity, through the above proof process, we find that Algorithm 2 only visits $O(\frac{1}{\alpha\epsilon})$ nodes, so it only requires $O(\frac{1}{\alpha\epsilon})$ space to maintain r , π , Q , and D . Hence, the total space complexity of Algorithm 2 is $O(\frac{1}{\alpha\epsilon})$. $\square$

Remark. While the proposed *HLA* lacks a formal guarantee of accuracy (the practical performance of *HLA* is comparable to the baselines in our empirical results), it boasts an outstanding characteristic in terms of *local* time complexity [66]. Specifically, *HLA* operates with a runtime that scales in direct proportion to the size of the cluster it generates, irrespective of the total size of the input graph. This attribute renders it exceptionally well-suited for the analysis of massive graphs.

V. EXPERIMENTAL EVALUATION

We conduct extensive experiments to test the effective and efficiency of the proposed solutions. All experiments are conducted on a Linux server with one Intel(R) Xeon(R) Silver 4210 2.20GHz CPU and 1 TB RAM.

A. Experimental Setup

Datasets. We evaluate our solutions on eight publicly-available datasets (Table IV), which are widely used benchmarks for

temporal motif analysis and temporal graph clustering [18], [20], [21], [42], [53], [59], [67]–[70]. Specifically, *CM* (CollegeMsg) contains message exchanges on a Facebook-like platform at UC Irvine. *EE* (email-Eu-core-temporal) consists of email interactions among users within a European research institution. *SA* (SMS-A) contains the short message interaction among users. *FW* (facebook-wall) is the wall posts between users on the social network Facebook located in the New Orleans region. *MO* (sx-mathoverflow), *UB* (sx-askubuntu), *SU* (sx-superuser), and *SO* (sx-stackoverflow) are temporal interaction networks between users on Math Overflow, Ask Ubuntu, Super User, and Stack Overflow, in terms of comments, questions, and answers. All datasets can be downloaded from <https://networkrepository.com/> and <https://snap.stanford.edu/data/>. Note that the self-loops are deleted, and the largest weakly connected components of these datasets are used in the experiments.

Competitors. *AGA* (Algorithm 1) and *HLA* (Algorithm 2) are our proposed methods. Besides, the following ten well-known competitors are implemented for comparison. (1) Lower-order temporal graph clustering: *TSC* [8], *TMO* [9], and *TDS* [11]. *TSC* is rooted in spectral clustering, whereas *TMO* and *TDS* are based on modularity optimization and density optimization on temporal networks, respectively; (2) Higher-order static graph clustering: *HSC* [37]–[39], *HMO* [54], [55], and *HDS* [56]–[58]. *HSC*, *HMO* and *HDS* apply higher-order spectral clustering, modularity optimization and density optimization to the de-temporal graph, respectively; (3) Higher-order temporal graph clustering: *THDS* [53], *THSC* [38], *THLC* [36], and *THPR* [28]. *THDS* optimizes temporal motif density, while the remaining three optimize temporal motif conductance (Definition 7). We employ the SOTA exact temporal motif mining algorithm² proposed by Mackey et al. [41] for counting δ -instances, which is more general and efficient compared to other methods [42], [43]. The core innovation of our paper lies not in making temporal motif counting faster, but in a deliberate redesign that replaces the existing enumeration-based approach with our proposed novel counting-based clustering framework. This design is intentionally forward-compatible, allowing for the seamless adoption of more efficient counting techniques as they emerge.

Parameters. The six temporal motifs (illustrated in Figure 4) we utilize are broadly representative across most applications [18], [20], [21], [42], [53], [59], [67]–[69]. For example, M_3 represents a temporal path involving three consecutive interactions at four different nodes. This motif is fundamental for understanding the flow of information or influence propagation in temporal networks, such as the spread of rumors in social media or transaction chains in financial systems. Clusters enriched with such motifs can reveal chains of influence or potential spreading cascades. M_5 represents a more complex and branched temporal chain structure. In particular, M_5 can

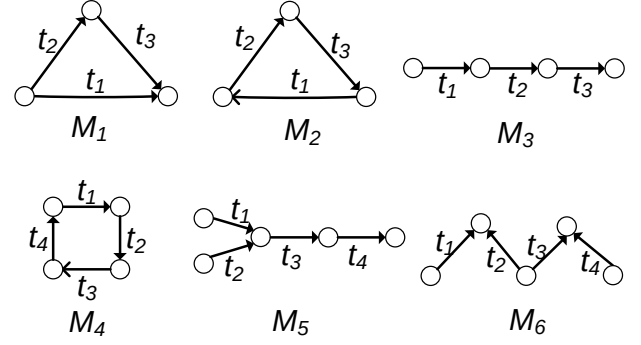


Fig. 4: Temporal motifs used for experimental evaluation. $t_1 \leq t_2 \leq t_3 \leq t_4$. The temporal duration δ is treated as a parameter.

represent scenarios in which two different individuals contact a key person who then relays or distributes information to others. Thus, in organizational or collaboration networks, M_5 may reflect task assignment or escalation patterns. In fraud or cybersecurity, M_5 can reveal coordinated attempts converging on a target, followed by subsequent suspicious activity. By including M_5 , our approach can cluster these branching and multi-source temporal flows. These motifs encompass various topologies, including triangles, lines, cycles, and more intricate patterns, with the largest motifs presenting particularly challenging inputs. Following [18], [41], [42], [59], we set M_2 as the query temporal motif and δ to one hour by default. Due to constraints related to extended running times and limited space, our evaluation is confined to these specific motifs. Nevertheless, it is important to note that our algorithms are versatile and can be readily applied to any temporal motif without modification.

Unless specified otherwise, we set $\alpha = 0.2$ and $\epsilon = \frac{1}{m}$ as the default standard settings [48], [66], [71]. Moreover, given that both *THLC* and *HLA* require a source vertex as input, we ensure robustness by randomly selecting five vertices to serve as source vertices and then report the average running time and clustering qualities. For the other baselines, we use the default parameters recommended by the authors.

B. Clustering Efficiency Evaluation

Since the objective functions of *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, *HDS*, and *THDS* are different from the temporal motif conductance studied in this work, comparing their efficiency is neither relevant nor essential. So, we exclude them for brevity.

Exp-1: Running time of different temporal motif conductance algorithms. Figure 5 shows the running time of *THSC*, *THLC*, *THPR*, *AGA*, and *HLA* with varying temporal motifs, with analogous trends observed across different δ (details in [72]). As can be seen, we have: (1) *HLA* is consistently faster than other methods because it is a pure local algorithm. Unlike alternatives, it does not precompute the weight $(W_{ef})_{uv}$ for every edge (u, v) ; instead, it calculates $(W_{ef})_{uv}$ only upon accessing the edge (u, v) . Consequently, *HLA*'s runtime scales linearly with the size of the cluster it generates, independent of the entire graph's size (Table II). (2) *AGA*

²Although approximation algorithms [20], [21], [49], [50] can improve computational efficiency, analyzing their theoretical guarantees for clustering quality remains challenging and will be considered as future work.

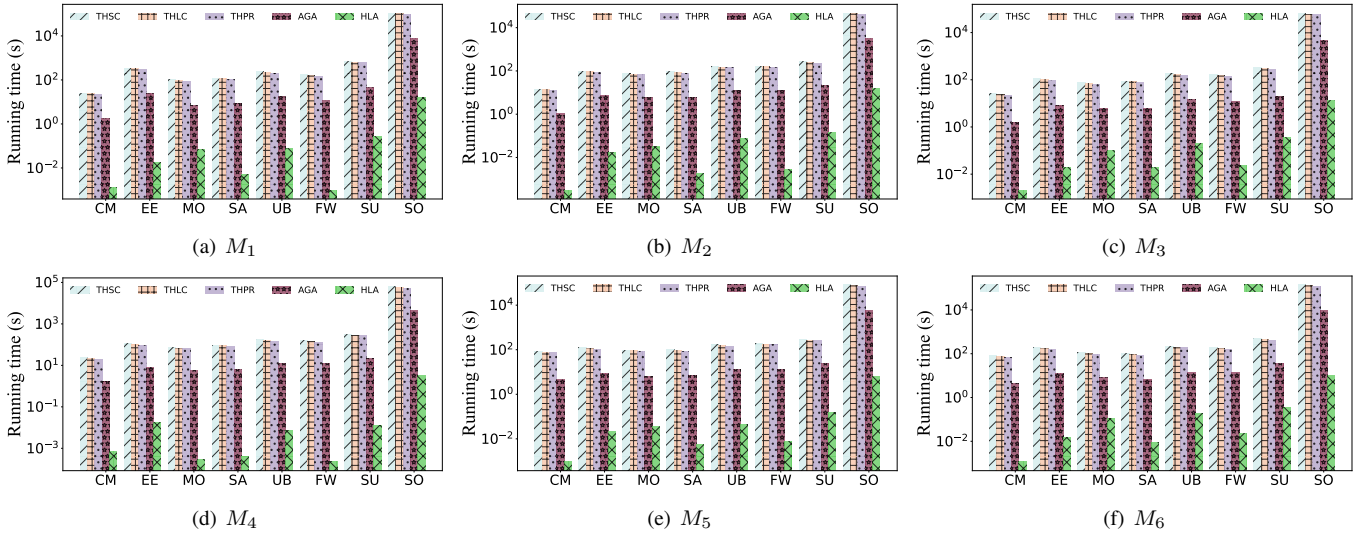


Fig. 5: Running time of different temporal motif conductance algorithms with varying temporal motif.

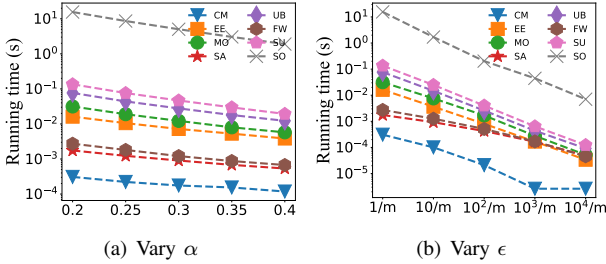


Fig. 6: Running time of *HLA* with varying parameters.

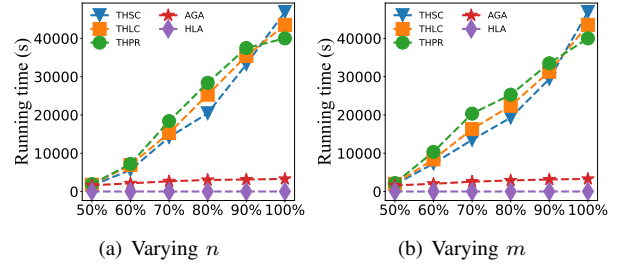


Fig. 7: Scalability testing.

is the runner-up on all datasets and all temporal motifs. The efficiency of *AGA* can be attributed to its novel edge-focused weighted graph projection method, which can be efficiently implemented using the powerful temporal motif counting paradigm. However, *THSC*, *THLC*, and *THPR* depend on the exhaustive enumeration of all temporal motif instances, making them significantly slower (Table II). In particular, our algorithms (i.e., *AGA* and *HLA*) achieve the speedups of $1 \sim 4$ orders of magnitude over these enumeration-based methods in most cases. For example, on *SO* and M_6 , *AGA* and *HLA* take 9154 and 10 seconds to obtain the result, while *THSC*, *THLC*, and *THPR* require 139510, 126581, 121248 seconds, respectively. These results give preliminary evidence that the proposed solutions are indeed highly efficient in practice.

Exp-2: Running time of *HLA* with varying parameters.

Figure 6 illustrates how the parameters α and ϵ affect the running time of *HLA* with M_2 as the query temporal motif and $\delta = 1$ hour. Other query temporal motifs and δ can also obtain similar results. As can be seen, we have (1) the running time of *HLA* decreases with increasing α . An intuitive explanation is that when α increases, each vertex u that needs to transmit information will pass on a smaller residual amount to its neighbors (Line 25 of Algorithm 2), resulting in a lower probability for its neighbors to be executed in the next round of information transmission (Line 23 of of Algorithm 2). Thus, when α increases, fewer nodes are visited, resulting in the

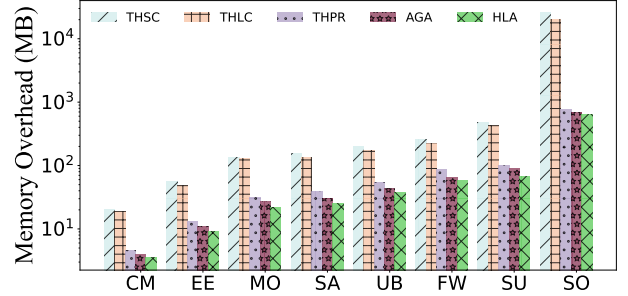


Fig. 8: Memory overhead (excluding the graph itself).

locality of *HLA* being stronger (details in Algorithm 2). (2) The running time of *HLA* decreases with increasing ϵ . An intuitive explanation is that when ϵ increases, each vertex u has a smaller probability of being able to transmit information (Line 23 of Algorithm 2), resulting in fewer neighbors being visited and stronger locality (details in Algorithm 2). Thus, these results align with our theoretical analysis (Table II).

Exp-3: Scalability testing. We further investigated the scalability of our approaches using eight synthetically generated graphs. Figure 7 shows the runtime for query motif M_2 at $\delta = 1$ hour (other settings exhibited similar patterns). These graphs are produced by sampling edges or nodes from the *SO* dataset, which has the highest edge volume, at percentages from 50% to 100%. The results confirm that our algorithms,

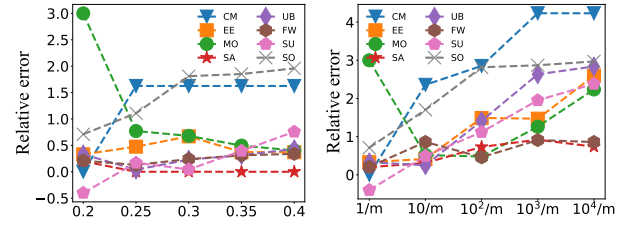
AGA and *HLA*, scale efficiently with increasing graph size. In contrast, the baselines show unstable performance, with runtimes that fluctuate significantly. This is attributable to their non-linear or exponential time complexity concerning n and m (see Table II), demonstrating that our methods are uniquely suited for massive graphs.

Exp-4: Memory overhead comparison. The memory consumption of all algorithms, evaluated with query motif M_2 and $\delta = 1$ hour, is compared in Figure 8; results for other parameter settings follow a consistent pattern. As theoretically predicted, our proposed algorithms, *AGA* and *HLA*, significantly reduce memory usage across all datasets, outperforming all baselines by up to an order of magnitude. A case in point is the *SO* dataset, where *AGA* and *HLA* use only 698 MB and 643 MB, respectively, in stark contrast to the substantial 25628 MB and 20226 MB required by *THSC* and *THLC*. This efficiency originates from our edge-focused weighted graph projection, which confines memory complexity to $O(m)$ by assigning weights solely to the original edges. Conversely, *THSC* and *THLC* incur a much higher cost of $O(\min n^2, \binom{k}{2} \text{num}(\mathcal{M}_\delta))$ due to their construction of hyperedges for motif instances. Furthermore, *HLA* proves slightly more efficient than *AGA* and *THPR*. This is because *THPR* maintains some auxiliary structures, such as temporal motif resident information for each vertex (Definition 9), and *AGA* requires additional space to store the intermediate variables $(W_{ef})(u, v)$ for all edge (u, v) ; instead, *HLA* only store $(W_{ef})_{uv}$ upon accessing the edge (u, v) . The results provide strong experimental validation for our theoretical memory complexity claims in Table II.

C. Clustering Quality Evaluation

Evaluating the quality of temporal clustering poses greater challenges than its static counterpart, primarily due to the absence of ground-truth clusters for temporal networks [8], [48], [70]. Fortunately, we can adopt the following two widely used temporal structure metrics [8], [44], [45], [47], [48], [70] for qualitative testing: temporal density (*TD*) and temporal conductance (*TC*). Specifically, let C be the identified cluster, the two metrics are defined as follows: $TD(C) = |\{(u, v, t) \in \mathcal{E} | u, v \in C\}| / |C|$ and $TC(C) = |\{(u, v, t) \in \mathcal{E} | u \in C, v \in \bar{C}\}| / \sum_{u \in C} |\{(u, v, t) \in \mathcal{E}\}|$. Clearly, *TD* computes the temporal density of the internal structure of the cluster, *TC* measures the temporal separability of the cluster. Thus, the larger $TD(C)$ (resp., smaller $TC(C)$) indicates the denser inside (resp., sparser outside) of C . Besides, we also evaluate the clustering quality using the proposed temporal motif conductance (*TMC*), as defined in Definition 7. As a consequence, smaller *TMC* and *TC* indicate higher clustering quality, while larger *TC* indicates higher clustering quality.

Exp-5: Clustering quality of various clustering methods. Since traditional lower-order temporal (or higher-order static) graph clustering methods (i.e., *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, and *HDS*) do not rely on temporal motifs, Table V only shows results for M_2 as the query temporal motif and $\delta = 1$ hour. The empirical results of other settings are placed in the [72] due



(a) Relative error with varying α (b) Relative error with varying ϵ
Fig. 9: Quality comparison between *AGA* and *HLA*.

to space limitations. For the *TMC* metric, we have (1) *AGA* outperforms all other methods on seven out of eight datasets. This superiority is attributed to *AGA*'s ability to identify clusters with a near-linear approximation ratio, whereas other methods provide poor or no theoretical guarantees (Table II). (2) *HLA* is superior to all the other methods except *AGA*. This is because *HLA* builds upon *AGA*; it maintains strong practical performance despite lacking a theoretical guarantee. (3) *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, and *HDS* exhibit poor clustering quality. This is because they ignore the crucial temporal information or higher-order information. For the *TC&TD* metric, we have: (1) the proposed algorithm *AGA* achieves the best overall performance without the situation where one metric is excellent but another is poor. (2) *TSC* has good *TC* value but has poor *TD* value. On the contrary, *TDS* has good *TD* value but poor *TC* value. This is because *TSC* merely considers the external temporal separateness but ignores the internal temporal density, *TDS* merely considers the internal temporal density but ignores the external temporal separateness. (3) Other methods have the worst performance. This is because either they are static methods, which fail to take into account temporal information, or they do not consider the temporal dense inside or the temporal sparse outside. In a nutshell, these results indicate that existing models cannot optimize our proposed objective function well, and our algorithm is much denser and more separable in terms of temporal feature than existing models.

Remark. Optimizing *TMC*, *TD*, and *TC* simultaneously is very challenging (or even impossible). The reasons are as follows. *TD* and *TC* are the objective functions optimized by *TDS* and *TSC*, respectively, so it is reasonable that they achieve the best performance on the corresponding metric. Similarly, *TMC* is the metric we proposed, thus our superior performance on it is also expected. However, as shown in Table V, *TD* and *TC* are generally contradictory, that is, no single method can achieve the optimal performance on both simultaneously. In contrast, our method attains the best overall performance across these two metrics (i.e., the differences from the top performer are very minimal in both metrics, whereas other methods often excel in only one metric while performing poorly in the other), a finding that aligns with similar observations reported in [48]. Thus, our model is a trade-off among them.

Exp-6: Quality comparison between *AGA* and *HLA*. Figure 9 compares the clustering quality of the approximate global

TABLE V: Clustering quality of various clustering methods. The best and second-best results in each metric are marked in **bold** and underlined, respectively. Smaller *TMC* and *TC* indicate higher clustering quality, while larger *TC* indicates higher clustering quality. The baseline is excluded if it cannot finish clustering within two days.

TMC/TC/TD	CM	EE	MO	SA	UB	FW	SU	SO
TSC	0.6/ 0.20 /2.66	0.45/ 0.20 /279.82	0.87/ 0.17 /2.80	0.81/0.41/24.07	0.69/ 0.03 /2.54	0.58/ 0.02 /227.0	0.97/ 0.02 /6.85	0.83/ 0.07 /2.00
TMO	0.56/0.85/28.71	0.29/0.42/656.38	0.75/ <u>0.31</u> /6.99	0.64/ 0.01 /59.63	0.71/0.28/3.16	0.52/0.19/29.03	0.70/0.23/2.52	0.61/0.23/2.95
TDS	0.67/0.83/ 243.61	0.54/0.43/ 4878.0	0.70/0.51/ 547.89	0.69/ 0.01 / 6063.0	0.71/0.55/ 119.09	0.52/0.24/197.68	0.69/0.53/ 153.47	0.67/0.49/ 535.03
HSC	0.71/0.89/9.03	0.79/0.66/10.46	0.82/0.89/52.68	0.68/0.69/23.58	0.95/0.81/64.79	0.96/0.47/163	0.91/0.46/46.82	out of time
HMO	0.74/0.88/2.34	0.75/0.72/112.30	0.72/0.88/56.11	0.77/0.59/21.06	0.94/0.73/114.06	0.72/0.52/35.11	0.79/0.76/30.04	out of time
HDS	0.76/0.88/30.21	0.72/0.67/45.18	0.59/0.86/117.35	0.87/0.71/47.08	0.95/0.66/67.08	0.95/0.67/114.15	0.92/0.73/36.56	out of time
THDS	<u>0.10</u> /0.85/44.22	0.13/0.40/801.33	0.22/0.90/229.6	0.68/0.47/203.0	0.24/0.92/45.15	0.50/0.39/ 890.66	0.27/0.92/32.71	0.30/0.99/16.33
THSC	0.01 /0.52/37.46	0.08/0.39/802.0	0.09/0.98/17.5	0.95/0.47/213.0	0.09/0.92/44.61	0.66/0.79/17.33	0.12/0.90/61.72	0.15/0.96/9.28
THLC	0.01 /0.85/38.28	0.05/0.36/139.77	0.12/0.87/150.33	0.95/0.46/190.0	0.06/0.92/47.01	0.64/0.515/83.2	0.17/0.95/22.78	0.32/ 0.80/115.46
THPR	0.25/0.83/28.5	0.06/0.38/304.06	0.12/0.95/51.48	0.95/0.91/20.0	0.06/0.91/42.86	0.66/0.79/17.33	0.05/0.972/11.41	0.22/0.85/16.66
AGA	0.01 /0.52/47.46	0.03 /0.34/805.0	0.03 /0.48/270.5	0.50 /0.37/223.0	0.03 /0.12/44.61	0.33 /0.09/227.33	0.05/0.20/63.72	0.07 /0.16/159.28
HLA	0.01 /0.86/20.70	<u>0.04</u> /0.39/250.12	0.12/0.75/260.31	0.60/0.42/162.85	0.04/0.85/92.47	0.40/ 0.15/199.31	0.03 /0.8/49.54	0.12/0.51/259.83

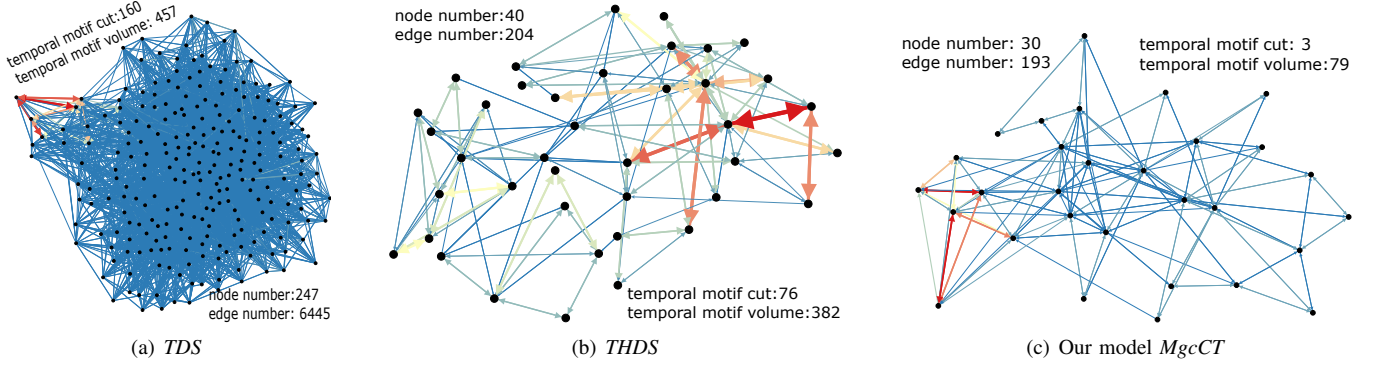


Fig. 10: Case studies. The thicker and darker the edge, the more temporal cycles it participates in.

algorithm (*AGA*) against the heuristic local algorithm (*HLA*). Using the temporal motif conductance ρ_{aga} from *AGA* as a benchmark, we define the relative error (RE) of *HLA* as $RE = (\rho_{hla} - \rho_{aga}) / \rho_{aga}$. As can be seen, we have (1) the RE is consistently bounded between -0.5 and 4, indicating the robust practical performance of *HLA*. (2) The RE generally increases with parameters α and ϵ . This trend is attributed to the role of these parameters in limiting the exploration scope of *HLA*; higher values restrict the search to a narrower neighborhood, increasing the risk of converging to a suboptimal local solution. Thus, the heuristic local algorithm *HLA* can identify high-quality clusters in practice.

Exp-7: Case studies. We conduct case studies on the Bitcoin dataset [73], which comprises 3,775 nodes and 24,180 temporal edges. Here, nodes represent users and temporal edges indicate when the corresponding users conducted the transaction through the well-known Bitcoin Alpha platform. To demonstrate the utility of our model, we select the temporal cycle M_4 in Figure 4 as the query motif, as it is a recognized indicator for identifying money laundering activities [23]–[25]. As summarized in Table I, we compare five temporal graph clustering methods (*TMO*, *TSC*, *TDS*, *THDS*, and *MgcCT*) with three static graph clustering methods (*HMO*, *HSC*, and *HDS*). Figure 10 visualizes the clusters identified by each method. Note that clusters detected by *TMO*, *TSC*, *HMO*, *HSC*, and *HDS* are either excessively large or too small for effective visualization. For instance, clusters identified by *TMO* and *HMO* contain over 1,000 vertices, reflecting the modularity

resolution limitation [74]. Furthermore, Table V confirms their poor performance in this context. In this case, *TDS* identifies a cluster of 247 users and 6,445 edges, densely connected via dyadic temporal edges but lacking higher-order temporal cycles (Figure 10(a)). Consequently, the number of temporal cycle instances is an order of magnitude lower than the number of edges. A detailed inspection of Figure 10(a) reveals that *TDS* fails to capture many money laundering transactions. Although *THDS* identifies more such transactions (Figure 10(b)), the resulting cluster exhibits numerous temporal cycle connections with nodes outside the cluster, violating the external sparsity condition and thus yielding an unsatisfactory result [16], [29]–[32]. In contrast, the cluster identified by our model *MgcCT* (Figure 10(c)) achieves a better balance of temporal cycles inside and outside the cluster. Specifically, the cluster identified by *MgcCT* contains more money laundering instances while minimizing cross-cluster money laundering activity. This suggests that the cluster identified by *MgcCT* is highly likely to represent organized financial crime groups [23], [69], [75]. Overall, these findings highlight the potential of our model for effective temporal graph clustering and its applicability in related domains.

VI. RELATED WORK

Static Graph Clustering. Graph clustering on static networks has received much attention over past decades [15]–[17]. Representative models include edge cut based (e.g., ratio cut or normalized cut [60]), null model based (e.g., modularity [76]), and subgraph cohesiveness based (e.g., clique or k -core [77]).

The above methods mainly focus on the internal or external dyadic edges of the cluster, resulting in that cannot capture the more informative and semantic higher-order structure. Until recently, motifs (i.e., small subgraphs) have been shown to be indispensable for modeling and understanding the higher-order organization of complex networks [38]. Thus, a wide spectrum of motif-based higher-order graph clustering methods have been proposed [33]–[39], [54]–[56], [58]. For example, motif modularity was proposed to extend the traditional modularity by optimizing the difference between the fraction of motif instances within the cluster and the fraction in a random network preserving the same degree of vertices [54], [55]. Higher-order densest subgraph model was proposed where the density is defined as the number of motif instances divided by the size of vertices [56]–[58]. However, all these methods cannot be directly applied to our problem due to they ignore the important time information.

Temporal Graph Clustering. Temporal dynamics constitute a critical dimension for modeling real-world systems, offering profound insights into graph clustering [8]–[11]. For example, Silva et al. [8] generalized sparsest and normalized cuts to temporal graphs by introducing temporal smoothness constraints on cut evolution. Mucha et al. [9] developed temporal modularity to quantify clustering quality across time slices, rewarding stable partitions with high instantaneous modularity and penalizing vertex migrations between clusters. Qin et al. [70] investigated bursting clusters in temporal networks, where clusters exhibit transient density peaks. Yu et al. [78] proposed an index-based method with pruning strategies to accelerate historical k -core discovery. However, these approaches are fundamentally limited by their focus on low-order connectivity patterns, rendering them inapplicable to our problem domain. Recently, Fu et al. [79], [80] studied the higher-order graph clustering on dynamic graphs, which aims to maintain the new higher-order clusters when vertices/edges are inserted/deleted. Thus, their objective function considers *only one timestamp*, while our model considers higher-order clusters that allow *span a time interval*. As a result, dynamic higher-order graph clustering [79], [80] is orthogonal to ours.

Temporal Motif Analysis. Temporal motifs have been widely recognized as fundamental higher-order structural units in temporal networks, with extensive research exploring their properties and applications [18], [20], [21], [23], [26], [49], [50], [81], [82]. Early foundational work by Kovanen et al. [81] introduced the first temporal motif definition that explicitly incorporates Δ -adjacency constraints between consecutive events (i.e., temporal edges), where two events are considered Δ -adjacent if their timestamp difference does not exceed Δ . Gurukar et al. [82] investigated communication motifs as recurring subgraphs with consistent information flow patterns, while Paranjape et al. [18] proposed the δ -temporal motif, which incorporates both temporal duration and ordering constraints (Definition 1). This work has inspired numerous exact algorithms for mining specific 3-edge δ -temporal motifs [18], [59], [67]–[69], as well as sampling-

based approximation methods [20], [21]. However, these small motifs face limitations in real-world applications [19], [41]–[43]. For arbitrary δ -temporal motifs with flexible node and edge configurations, Mackey et al. [41] developed the SOTA BT algorithm, an exact backtracking approach based on isomorphism testing. This has subsequently motivated hardware-accelerated implementations to optimize performance [42], [43]. Many randomized algorithms have been proposed for estimating motif counts [20], [21], [49], [50].

VII. CONCLUSION

To capture the interplay of continuous topological-temporal dependencies and higher-order structural patterns in temporal clusters, we introduce a novel metric, temporal motif conductance, quantifying cluster cohesion through motif-constrained random walk dynamics. We formalize our objective as minimizing this conductance and rigorously establish its NP-hardness. To address this computational challenge, we propose two complementary algorithms: a global approximation algorithm *AGA* achieving a near-optimal quadratic Cheeger bound for any temporal motif, and a scalable heuristic local algorithm *HLA* that bypasses full graph traversal, thereby achieving a superior balance between runtime efficiency and output quality. Extensive experiments on eight real-world temporal networks demonstrate that our framework outperforms ten competitors in both clustering efficiency and quality.

REFERENCES

- [1] J. Shi and J. Malik, “Normalized cuts and image segmentation,” in *CVPR*, 1997, pp. 731–737.
- [2] D. Tolliver and G. L. Miller, “Graph partitioning by spectral rounding: Applications in image segmentation and clustering,” in *CVPR*, 2006, pp. 1053–1060.
- [3] K. D. Devine, E. G. Boman, and G. Karypis, “Partitioning and load balancing for emerging parallel applications and architectures,” in *Parallel Processing for Scientific Computing*, 2006, pp. 99–126.
- [4] Z. Li, X. Jian, Y. Wang, and L. Chen, “CC-GNN: A community and contraction-based graph neural network,” in *ICDM*, X. Zhu, S. Ranka, M. T. Thai, T. Washio, and X. Wu, Eds., 2022, pp. 231–240.
- [5] F. M. Bianchi, D. Grattarola, and C. Alippi, “Spectral clustering with graph neural networks for graph pooling,” in *ICML*, 2020, pp. 874–883.
- [6] P. Holme, “Modern temporal network theory: A colloquium,” *CoRR*, vol. abs/1508.01303, 2015.
- [7] P. Holme and J. Saramaki, “Temporal networks,” *Physics Reports*, vol. 519, pp. 97–125, 2012.
- [8] A. Silva, A. K. Singh, and A. Swami, “Spectral algorithms for temporal graph cuts,” in *WWW*, 2018, pp. 519–528.
- [9] P. J. Mucha, T. Richardson, K. Macon, M. A. Porter, and J.-P. Onnela, “Community structure in time-dependent, multiscale, and multiplex networks,” *science*, vol. 328, no. 5980, pp. 876–878, 2010.
- [10] Y. Chi, X. Song, D. Zhou, K. Hino, and B. L. Tseng, “Evolutionary spectral clustering by incorporating temporal smoothness,” in *KDD*, 2007, pp. 153–162.
- [11] L. Oettershagen, A. L. Konstantinidis, and G. F. Italiano, “An edge-based decomposition framework for temporal networks,” in *WSDM*, 2025, pp. 735–743.
- [12] G. H. Nguyen, J. B. Lee, R. A. Rossi, N. K. Ahmed, E. Koh, and S. Kim, “Continuous-time dynamic network embeddings,” in *WWW*, 2018, pp. 969–976.
- [13] A. Çelikkanat, N. Nakis, and M. Mørup, “Continuous-time graph representation with sequential survival process,” in *AAAI*, 2024, pp. 11 177–11 185.
- [14] L. Han, X. Ma, L. Sun, B. Du, Y. Fu, W. Lv, and H. Xiong, “Continuous-time and multi-level graph representation learning for origin-destination demand prediction,” in *KDD*, 2022, pp. 516–524.

- [15] M. Maier, U. von Luxburg, and M. Hein, "Influence of graph construction on graph-based clustering measures," in *NIPS*, 2008, pp. 1025–1032.
- [16] J. Leskovec, K. J. Lang, and M. W. Mahoney, "Empirical comparison of algorithms for network community detection," in *WWW*, 2010, pp. 631–640.
- [17] S. Fortunato and D. Hric, "Community detection in networks: A user guide," *CoRR*, vol. abs/1608.00163, 2016.
- [18] A. Paranjape, A. R. Benson, and J. Leskovec, "Motifs in temporal networks," in *WSDM*, 2017, pp. 601–610.
- [19] P. Liu, V. Guarrasi, and A. E. Sariyüce, "Temporal network motifs: Models, limitations, evaluation," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 945–957, 2023.
- [20] P. Liu, A. R. Benson, and M. Charikar, "Sampling methods for counting temporal motifs," in *WSDM*, 2019, pp. 294–302.
- [21] J. Wang, Y. Wang, W. Jiang, Y. Li, and K. Tan, "Efficient sampling algorithms for approximate temporal motif counting," in *CIKM*, 2020, pp. 1505–1514.
- [22] L. Kovanen, K. Kaski, J. Kertész, and J. Saramäki, "Temporal motifs reveal homophily, gender-specific patterns, and group talk in call sequences," *Proceedings of the National Academy of Sciences*, vol. 110, no. 45, pp. 18070–18075, 2013.
- [23] R. Kumar and T. Calders, "2scent: An efficient algorithm to enumerate all simple temporal cycles," *Proc. VLDB Endow.*, vol. 11, no. 11, pp. 1441–1453, 2018.
- [24] M. Levi and P. Reuter, "Money laundering," *Crime and justice*, vol. 34, no. 1, pp. 289–375, 2006.
- [25] T. Squartini, I. van Lelyveld, and D. Garlaschelli, "Early-warning signals of topological collapse in interbank networks," *Scientific Reports*, vol. 3, 2013.
- [26] X. Cai, X. Ke, K. Wang, L. Chen, T. Zhang, Q. Liu, and Y. Gao, "Efficient temporal butterfly counting and enumeration on temporal bipartite graphs," *Proc. VLDB Endow.*, vol. 17, no. 4, pp. 657–670, 2023.
- [27] L. Lin, Z. Wang, R. Li, Q. Liu, H. Qin, and J. Zhao, "Provable higher-order graph clustering: The power of peeling-based approaches," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 9, pp. 5568–5583, 2025.
- [28] L. Lin, T. Jia, Z. Wang, J. Zhao, and R. Li, "PSMC: provable and scalable algorithms for motif conductance based graph clustering," in *KDD*, 2024, pp. 1793–1803.
- [29] R. Kamal and A. Bagchi, "A lovász-simonovits theorem for hypergraphs with application to local clustering," *Proc. ACM Manag. Data*, vol. 2, no. 4, pp. 190:1–190:27, 2024.
- [30] Y. He, L. Lin, P. Yuan, R. Li, T. Jia, and Z. Wang, "CCSS: towards conductance-based community search with size constraints," *Expert Syst. Appl.*, vol. 250, p. 123915, 2024.
- [31] L. Lin, Y. He, W. Chen, P. Yuan, R.-H. Li, and T. Jia, "Effective and Efficient Conductance-based Community Search at Billion Scale," *IEEE Transactions on Big Data*, no. 01, pp. 1–14. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TBDDATA.2025.3588028>
- [32] Y. Lou, C. Wang, T. Gu, H. Feng, J. Chen, and J. X. Yu, "Time-topology analysis," *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3322–3334, 2021.
- [33] A. R. Benson, D. F. Gleich, and J. Leskovec, "Tensor spectral clustering for partitioning higher-order network structures," in *SDM*, 2015, pp. 118–126.
- [34] T. Wu, A. R. Benson, and D. F. Gleich, "General tensor spectral co-clustering for higher-order data," in *NIPS*, D. D. Lee, M. Sugiyama, U. von Luxburg, I. Guyon, and R. Garnett, Eds., 2016, pp. 2559–2567.
- [35] D. Zhou, S. Zhang, M. Y. Yildirim, S. Alcorn, H. Tong, H. Davulcu, and J. He, "A local algorithm for structure-preserving graph cut," in *KDD*, 2017, pp. 655–664.
- [36] H. Yin, A. R. Benson, J. Leskovec, and D. F. Gleich, "Local higher-order graph clustering," in *KDD*, 2017, pp. 555–564.
- [37] C. E. Tsourakakis, J. Pachocki, and M. Mitzenmacher, "Scalable motif-aware graph clustering," in *WWW*, R. Barrett, R. Cummings, E. Agichtein, and E. Gabrilovich, Eds., 2017, pp. 1451–1460.
- [38] A. R. Benson, D. F. Gleich, and J. Leskovec, "Higher-order organization of complex networks," *Science*, vol. 353, no. 6295, pp. 163–166, 2016.
- [39] S. Huang, Y. Li, Z. Bao, and Z. Li, "Towards efficient motif-based graph partitioning: An adaptive sampling approach," in *ICDE*, 2021, pp. 528–539.
- [40] M. Meila and J. Shi, "A random walks view of spectral segmentation," in *AISTATS*, T. S. Richardson and T. S. Jaakkola, Eds., 2001.
- [41] P. Mackey, K. Porterfield, E. Fitzhenry, S. Choudhury, and G. Chin, "A chronological edge-driven approach to temporal subgraph isomorphism," in *IEEE BigData*, 2018, pp. 3972–3979.
- [42] N. Talati, H. Ye, S. Vedula, K.-Y. Chen, Y. Chen, D. Liu, Y. Yuan, D. Blaauw, A. Bronstein, T. Mudge *et al.*, "Mint: An accelerator for mining temporal motifs," in *MICRO*, 2022, pp. 1270–1287.
- [43] Y. Yuan, H. Ye, S. Vedula, W. Kaza, and N. Talati, "Everest: Gpu-accelerated system for mining temporal motifs," *Proc. VLDB Endow.*, vol. 17, no. 2, pp. 162–174, 2023.
- [44] L. Lin, P. Yuan, R. Li, and H. Jin, "Mining diversified top- k lasting cohesive subgraphs on temporal networks," *IEEE Trans. Big Data*, vol. 8, no. 6, pp. 1537–1549, 2022.
- [45] L. Lin, P. Yuan, R. Li, J. Wang, L. Liu, and H. Jin, "Mining stable quasi-cliques on temporal networks," *IEEE Trans. Syst. Man Cybern. Syst.*, vol. 52, no. 6, pp. 3731–3745, 2022.
- [46] Y. Zhang, L. Lin, P. Yuan, and H. Jin, "Significant engagement community search on temporal networks," in *DASFAA*, 2022, pp. 250–258.
- [47] C. Zhu, L. Lin, P. Yuan, and H. Jin, "Discovering cohesive temporal subgraphs with temporal density aware exploration," *J. Comput. Sci. Technol.*, vol. 37, no. 5, pp. 1068–1085, 2022.
- [48] L. Lin, P. Yuan, R. Li, C. Zhu, H. Qin, H. Jin, and T. Jia, "Qtcs: Efficient query-centered temporal community search," *Proc. VLDB Endow.*, vol. 17, no. 6, pp. 1187–1199, 2024.
- [49] I. Sarpe and F. Vandin, "oden: Simultaneous approximation of multiple motif counts in large temporal networks," in *CIKM*, 2021, pp. 1568–1577.
- [50] —, "Presto: Simple and scalable sampling techniques for the rigorous approximation of temporal motif counts," in *SDM*, 2021, pp. 145–153.
- [51] H. Wu, Y. Zhao, J. Cheng, and D. Yan, "Efficient processing of growing temporal graphs," in *Proceedings of DASFAA*, 2017.
- [52] W. Ahmad, M. A. Porter, and M. Beguerisse-Díaz, "Tie-decay networks in continuous time and eigenvector-based centralities," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1759–1771, 2021.
- [53] I. Sarpe, F. Vandin, and A. Gionis, "Scalable temporal motif densest subnetwork discovery," in *KDD*, 2024, pp. 2536–2547.
- [54] L. Huang, H. Chao, and G. Xie, "Mumod: A micro-unit connection approach for hybrid-order community detection," in *AAAI*, 2020, pp. 107–114.
- [55] A. Arenas, A. Fernandez, S. Fortunato, and S. Gomez, "Motif-based communities in complex networks," *Journal of Physics A: Mathematical and Theoretical*, vol. 41, no. 22, p. 224001, 2008.
- [56] Y. Fang, K. Yu, R. Cheng, L. V. S. Lakshmanan, and X. Lin, "Efficient algorithms for densest subgraph discovery," *Proc. VLDB Endow.*, vol. 12, no. 11, pp. 1719–1732, 2019.
- [57] Y. He, K. Wang, W. Zhang, X. Lin, and Y. Zhang, "Scaling up k-clique densest subgraph detection," *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 69:1–69:26, 2023.
- [58] B. Sun, M. Danisch, T. H. Chan, and M. Sozio, "Kclist++: A simple algorithm for finding k-clique densest subgraphs in large graphs," *Proc. VLDB Endow.*, vol. 13, no. 10, pp. 1628–1640, 2020.
- [59] J. Li, J. Qi, Y. Huang, L. Cao, Y. Yu, and J. Dong, "Motto: Scalable motif counting with time-aware topology constraint for large-scale temporal graphs," in *CIKM*, 2024, pp. 1195–1204.
- [60] U. Von Luxburg, "A tutorial on spectral clustering," *Statistics and computing*, vol. 17, no. 4, pp. 395–416, 2007.
- [61] L. Lin, R. Li, and T. Jia, "Scalable and effective conductance-based graph clustering," in *AAAI*, 2023, pp. 4471–4478.
- [62] D. Zhou, J. He, H. Davulcu, and R. Maciejewski, "Motif-preserving dynamic local graph cut," in *Big Data*, 2018, pp. 1156–1161.
- [63] G. H. Golub and C. F. Van Loan, *Matrix computations*. JHU press, 2013.
- [64] N. Alon and V. D. Milman, " λ_1 , isoperimetric inequalities for graphs, and superconcentrators," *Journal of Combinatorial Theory, Series B*, vol. 38, no. 1, pp. 73–88, 1985.
- [65] M. Fiedler, "Algebraic connectivity of graphs," *Czechoslovak mathematical journal*, vol. 23, no. 2, pp. 298–305, 1973.
- [66] R. Andersen, F. R. K. Chung, and K. J. Lang, "Local graph partitioning using pagerank vectors," in *FOCS*, 2006, pp. 475–486.
- [67] N. Pashanasangi and C. Seshadhri, "Faster and generalized temporal triangle counting, via degeneracy ordering," in *KDD*, 2021, pp. 1319–1328.
- [68] Z. Gao, C. Cheng, Y. Yu, L. Cao, C. Huang, and J. Dong, "Scalable motif counting for large-scale temporal graphs," in *ICDE*, 2022, pp. 2656–2668.

- [69] Y. Xia, Y. Fang, and W. Luo, "Efficiently counting triangles in large temporal graphs," *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 1–27, 2025.
- [70] H. Qin, R. Li, Y. Yuan, G. Wang, L. Qin, and Z. Zhang, "Mining bursting core in large temporal graph," *Proc. VLDB Endow.*, vol. 15, no. 13, pp. 3911–3923, 2022.
- [71] Z. Wei, X. He, X. Xiao, S. Wang, S. Shang, and J. Wen, "Topppr: Top-k personalized pagerank queries with precision guarantees on large graphs," in *SIGMOD*, 2018, pp. 441–456.
- [72] Anonymous, "full version available at: <https://anonymous.4open.science/r/MgcCT-B04F>," 2025.
- [73] S. Kumar, B. Hooi, D. Makhija, M. Kumar, C. Faloutsos, and V. Subrahmanian, "Rev2: Fraudulent user prediction in rating platforms," in *WSDM*, 2018, pp. 333–341.
- [74] S. Fortunato and M. Barthélemy, "Resolution limit in community detection," *Proceedings of the national academy of sciences*, vol. 104, no. 1, pp. 36–41, 2007.
- [75] P. Liu, R. Acharyya, R. E. Tillman, S. Kimura, N. Masuda, and A. E. Sariyuce, "Temporal motifs for financial networks: A study on mercari, jpmc, and venmo platforms," *CoRR*, vol. abs/2301.07791, 2023.
- [76] M. E. Newman, "Fast algorithm for detecting community structure in networks," *Physical review E*, vol. 69, no. 6, p. 066133, 2004.
- [77] L. Chang and L. Qin, "Cohesive subgraph computation over large sparse graphs," in *Proceedings of ICDE*, 2019, pp. 2068–2071.
- [78] M. Yu, D. Wen, L. Qin, Y. Zhang, W. Zhang, and X. Lin, "On querying historical k-cores," *Proc. VLDB Endow.*, vol. 14, no. 11, p. 2033–2045, 2021.
- [79] D. Fu, D. Zhou, and J. He, "Local motif clustering on time-evolving graphs," in *SIGKDD*, 2020, pp. 390–400.
- [80] D. Fu, D. Zhou, R. Maciejewski, A. Croitoru, M. Boyd, and J. He, "Fairness-aware clique-preserving spectral clustering of temporal graphs," in *WWW*, 2023, pp. 3755–3765.
- [81] L. Kovanen, M. Karsai, K. Kaski, J. Kertész, and J. Saramäki, "Temporal motifs in time-dependent networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2011, no. 11, p. P11005, 2011.
- [82] S. Gurukar, S. Ranu, and B. Ravindran, "COMMIT: A scalable approach to mining communication motifs from dynamic networks," in *SIGMOD*, 2015, pp. 475–489.

A. Additional Experiments

Running time of different temporal motif conductance algorithms with varying δ . Figure 11 shows the running time of *THSC*, *THLC*, *THPR*, *AGA*, and *HLA* with varying the temporal duration δ . As can be seen, we have (1) the consistent speed advantage of *HLA* stems from its purely local design. Rather than precomputing the weight $(W_{ef})_{uv}$ for all edges (u, v) in advance like other approaches, it calculates $(W_{ef})_{uv}$ on the fly when edge (u, v) is accessed. As a result, the runtime of *HLA* increases linearly with the size of the generated cluster, regardless of the overall graph scale (Table II). (2) Across all tested datasets and δ values, *AGA* consistently ranks as the second best performing method. Its efficiency is largely due to an innovative edge-centric weighted graph projection technique, which can be implemented effectively through the well-established temporal motif counting framework. In contrast, methods such as *THSC*, *THLC*, and *THPR* rely on exhaustively enumerating every temporal motif instance, leading to substantially longer execution times (Table II). (3) All algorithms exhibit relatively stable performance as δ increases. This is because their time complexity mainly comes from the enumeration or counting of the temporal motif instance, but when δ increases within a relatively small range, it will not significantly affect their efficiency. In a nutshell, these results give preliminary evidence that the proposed

solutions are indeed highly efficient in practice.

Effectiveness of different temporal motif conductance algorithms with varying parameters. Based on the experimental results presented in Figures 12 and 13, the following key observations can be drawn: (1) The proposed algorithms, *AGA* and *HLA*, demonstrate consistent superiority over existing baseline methods across a wide range of experimental configurations, including diverse temporal motif structures and varying temporal duration δ . Notably, despite the absence of theoretical guarantees, *HLA* achieves performance levels comparable to those of *AGA* in most scenarios. This suggests that the practical efficacy of *HLA* remains robust, highlighting its value in real-world applications where theoretical optimality may be secondary to empirical performance. (2) On average, the performance of our proposed *AGA* and *HLA* algorithms exhibit relative stability. This stability is evident across different datasets, temporal motifs, and temporal durations δ . In contrast, the performance of other baseline methods displays considerable variability, which is highly dependent on the specific characteristics of the dataset, temporal motif, and temporal duration δ . These findings provide further empirical evidence supporting the notion that our algorithms are adept at identifying clusters of higher quality compared to existing baseline methods when applied to real-world graph structures. This empirical validation aligns closely with the theoretical analysis presented in Table II, reinforcing the theoretical underpinnings of our proposed methodologies.

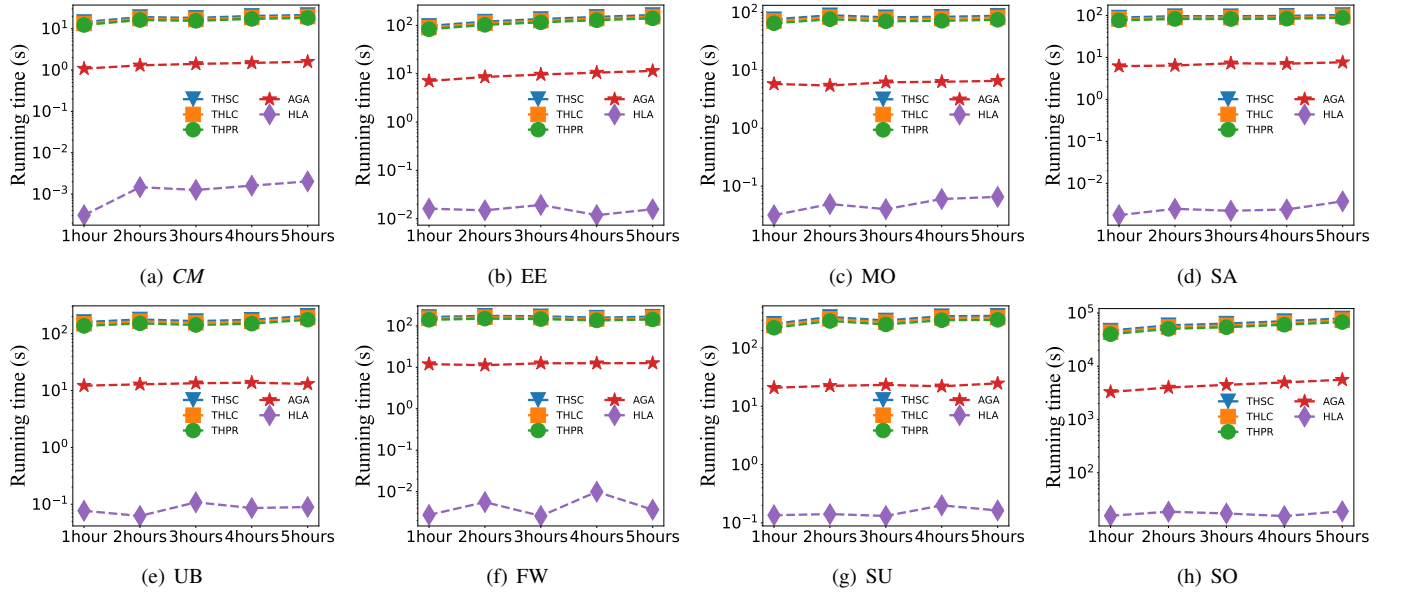


Fig. 11: Running time of different temporal motif conductance algorithms with varying δ .

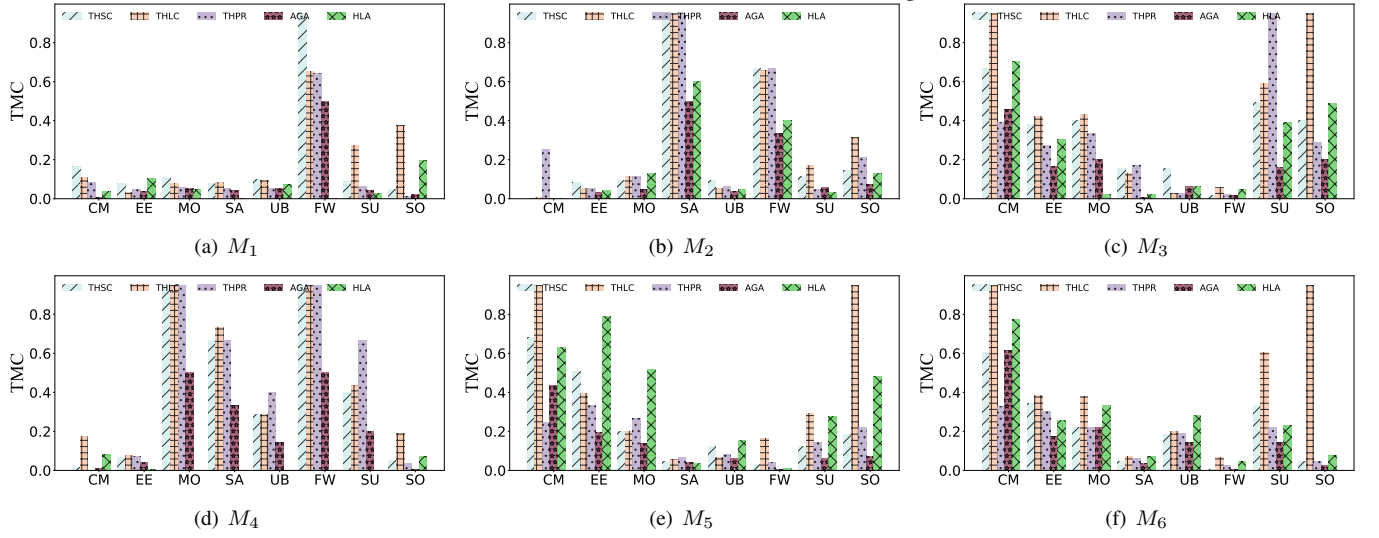


Fig. 12: Effectiveness of different temporal motif conductance with varying temporal motif.

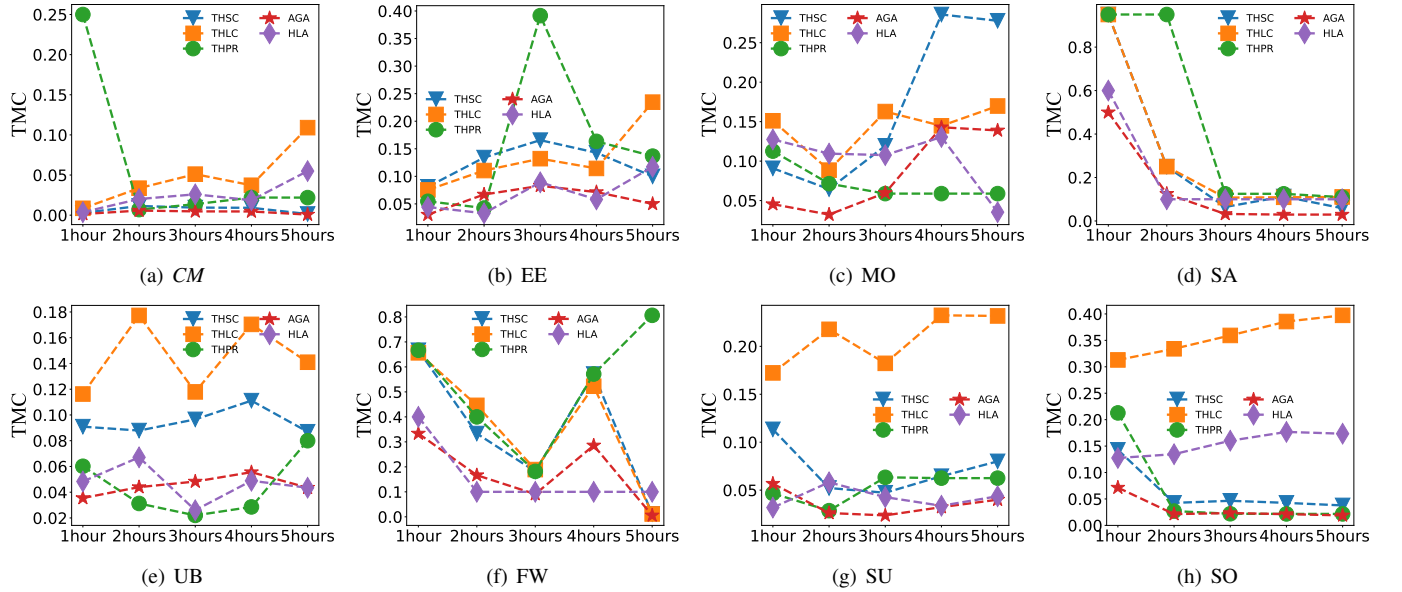


Fig. 13: Effectiveness of different temporal motif conductance with varying δ .